
VideoGenDoctor: Structured Failure Diagnosis, Evidence Localization, and Repair Planning for Controllable Video Generation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Controllable video generation increasingly relies on structured conditions such as character identity, camera trajectory, required props, temporal actions, and multi-shot specifications. Existing evaluation pipelines mostly return scalar quality or alignment scores, which are insufficient for debugging because they rarely identify what failed, where it failed, which evidence supports the diagnosis, or how the next generation attempt should be repaired.

We present VideoGenDoctor, a structured framework for failure diagnosis, evidence localization, and repair planning. Given a generated video and its prompt or structured specification, VideoGenDoctor converts failures into code-span-target-plan records grounded in temporal evidence and representative keyframes. The default implementation combines lightweight feature-based screening, optional VLM verification, and a patch compiler that maps localized failures to repair plans.

We position VideoGenDoctor-Bench-v0 as a controlled diagnostic fixture rather than a broad open-world benchmark. It contains 324 videos, 9 deterministic perturbation types, and 2,016 human-verified evidence spans covering 12 observed failure codes from a 38-code taxonomy. On this fixture, VideoGenDoctor-diagnosis reaches macro-F1 0.9110 and tIoU@0.5 0.7316; Rule+GPT-4V reaches 0.9276 and 0.7688. In automatic closed-loop evaluation, VideoGenDoctor-full improves Pass@1/2 from 24.07%/35.80% for Score-only feedback to 64.81%/83.64%. Rule+GPT-4V + repair loop remains the strongest absolute scorer on the controlled fixture, but at $2.75\times$ relative cost; we therefore treat VideoGenDoctor-full as the default operating point for the main narrative and use the GPT-4V front end as a high-cost reference. On a 240-video real-failure subset from four controllable video generators, the default VideoGenDoctor pipeline improves front-end diagnosis macro-F1 from 0.7992 to 0.8264 and tIoU@0.5 from 0.5431 to 0.5982 over direct structured VLM reporting, while also improving downstream repair utility. These results support the value of structured, auditable repair records while leaving broad open-world generality as future validation.

1 Introduction

Recent diffusion-based models can synthesize multi-second videos conditioned on text, reference images, camera trajectories, character identities, and structured shot scripts Yang et al. [2024], Blattmann et al. [2023], Wang et al. [2023], Yuan et al. [2024]. In production-style workflows, ShotIR-like specifications define per-shot props, characters, camera movement, shot type, and action

order. When a generated video violates such specifications, however, users still lack a systematic interface for locating the failure and deciding the next intervention.

Current video-generation evaluators such as VBench Huang et al. [2024], EvalCrafter Liu et al. [2024], VideoScore He et al. [2024], and T2V-CompBench Sun et al. [2025] report aggregate quality, motion, or alignment scores. These scores are useful for model comparison but weak as debugging signals: a low score does not indicate whether the character identity drifted, a required prop disappeared, camera motion was violated, or a specific temporal segment should be regenerated.

VideoGenDoctor reframes controllable-video evaluation as structured diagnosis and repair planning. Given a video V and specification S , it outputs records

$$R = \{(c_i, t_0^i, t_1^i, y_i, \sigma_i, K_i, T_i, a_i)\},$$

where c_i is a failure code, (t_0^i, t_1^i) is the evidence span, y_i is the affected target, σ_i is confidence, K_i contains representative keyframes, T_i stores optional track-level evidence, and a_i is a compiled repair action. The interface is designed to be machine-readable, human-auditable, and compatible with downstream video repair tools.

Scope. We separate the interface claim from the open-world generality claim. The expanded controlled fixture supports the claim that code–span–target–plan records provide auditable diagnosis and useful repair plans under known perturbations. A real-failure subset suggests transfer beyond deterministic perturbations, but broader coverage across generators, domains, styles, and long-form videos remains future work. We therefore call VideoGenDoctor-Bench-v0 a controlled diagnostic fixture rather than a general open-world benchmark.

Contributions. This paper contributes: (1) a code–span–target–plan representation for controllable video debugging; (2) a modular framework combining feature-based candidate generation, optional VLM verification, evidence aggregation, and patch compilation; (3) VideoGenDoctor-Bench-v0, with 324 controlled videos, 2,016 verified evidence spans, 9 perturbation types, and 12 observed codes, plus a 240-video real-failure subset covering 18 observed codes; and (4) an evaluation against direct VLM diagnosis, VLM-to-patch repair, blind human repair validation, per-code reliability analysis, paired bootstrap comparisons, and real-failure transfer checks.

2 Related Work

Video generation evaluation. Existing benchmarks and evaluators measure aggregate video quality, motion smoothness, subject consistency, and text-video alignment Huang et al. [2024], Liu et al. [2024], He et al. [2024], Sun et al. [2025]. They are valuable for ranking models but do not usually produce localized evidence or executable repair plans. VideoGenDoctor instead returns structured failure records tied to temporal spans and repair templates. A compact capability comparison is provided in Appendix A, Table 9.

VLM critics and temporal grounding. Large vision-language models can inspect sampled frames and produce critiques Liu et al. [2023], Chen et al. [2024], OpenAI [2023]. We therefore treat direct VLM reporting and VLM-to-patch generation as strong baselines rather than strawmen. The central question is whether constraining critique through code–span–target records improves temporal grounding, schema validity, auditability, repair executability, and cost. For temporal localization, we adapt temporal intersection-over-union:

$$\text{tIoU} = \frac{\max(0, \min(\hat{t}_1, t_1^*) - \max(\hat{t}_0, t_0^*))}{\max(\hat{t}_1, t_1^*) - \min(\hat{t}_0, t_0^*)}.$$

Automated repair. Program repair systems typically localize a defect, classify it, and generate a patch. VideoGenDoctor adapts this pattern to controllable video generation: each diagnosis must expose evidence and map to a supported repair family, connecting evaluation directly to iterative generation.

77 3 VideoGenDoctor

78 3.1 Overview

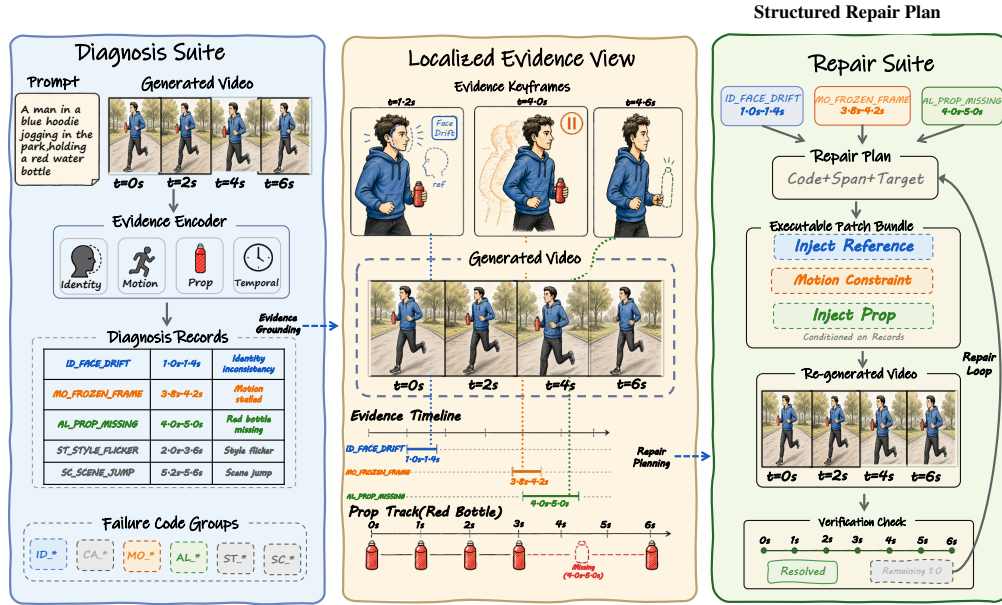


Figure 1: VideoGenDoctor framework overview. The system converts a generated video and its prompt or structured specification into failure-as-code diagnosis records, grounds each record in localized temporal evidence, maps records to structured repair plans, and verifies regenerated videos in a closed loop.

79 VideoGenDoctor contains four modules: segmentation and feature extraction, candidate diagnosis,
80 optional VLM verification, and patch compilation. The first module samples keyframes and computes
81 lightweight signals; the second applies thresholded rules to produce failure candidates; the third
82 calibrates top-ranked candidates using structured VLM questions; and the fourth maps retained
83 diagnosis records to structured repair actions such as reference injection, prop injection, camera
84 override, segment regeneration, or style correction.

85 3.2 Failure Records and Patch Templates

86 VideoGenDoctor uses a machine-readable taxonomy with 38 codes grouped into Identity, Camera,
87 Motion, Alignment, Style, and Scene. The current controlled fixture evaluates 12 observed codes; the
88 remaining codes define an extensible namespace and are not treated as empirically validated. The full
89 taxonomy and group-level description are provided in Appendix A and Appendix C.

90 Each predicted failure is represented as

$$r = (c, t_0, t_1, y, \tilde{\sigma}, K, T, a),$$

91 where c is the taxonomy code, (t_0, t_1) is the evidence span, y is the affected target, $\tilde{\sigma}$ is confidence,
92 K stores representative keyframes, T stores optional tracks, and a is the compiled repair action.
93 Detectors and judges can be replaced without changing this schema.

Table 1: Representative failure types and repair-plan template mappings. A diagnosis record combines a failure type with a localized span, target, confidence, evidence, and repair-plan action.

Failure type	Evidence signal	Repair action
Face identity drift	Face embedding drift across keyframes	Reference-frame injection
Body / clothing drift	Character-region appearance drift	Reference-frame or character-anchor injection
Camera movement deviation	Camera flow trace or trajectory mismatch	Camera movement override
Frozen frames or action stalls	Near-zero flow or stalled pose track	Regenerate affected segment
Missing required prop	Object presence check or VLM prop-absence judgment	Prop injection
Style inconsistency	Style or color embedding drift across segments	Style correction
Background inconsistency	Scene/background embedding drift	Background anchoring

94 3.3 Candidate Diagnosis and Verification

95 Given video V , we partition it into 2-second fixed-stride segments and sample six keyframes per
 96 segment. The default implementation computes semantic embedding drift with OpenCLIP Cherti
 97 et al. [2023], optical flow with RAFT Teed and Deng [2020] or OpenCV Farneback Bradski [2000],
 98 Farneback [2003], face drift with InsightFace Deng et al. [2019], and object or prop cues with
 99 YOLOv8 Ultralytics [2023] when enabled. For each segment s and failure code c , rule ρ_c produces
 100 a score $\sigma_{s,c} \in [0, 1]$; top-ranked candidates inherit segment boundaries and keyframes ranked by
 101 frame-level anomaly scores.

102 The optional Stage-2 judge receives the top- K candidate segments ($K = 3$ by default), sampled
 103 keyframes, temporal bounds, candidate codes, and relevant prompt or ShotIR fields. It answers
 104 structured questions rather than writing an unconstrained report. Final confidence is

$$\tilde{\sigma}_{s,c} = (1 - \alpha)\sigma_{s,c}^{(1)} + \alpha\sigma_{s,c}^{(2)},$$

105 with $\alpha = 0.6$ by default. Hosted endpoints, prompts, sampled frames, raw responses, parsed JSON,
 106 and latency metadata are logged for reproducibility.

107 3.4 Evidence Localization and Repair Planning

108 Each retained record stores an evidence object $E = (t_0, t_1, K, T)$ so that the diagnosis can be audited.
 109 Fixed segments are sufficient for the current fixture; future versions can refine spans using frame-level
 110 peaks and track-based merging. The patch compiler then emits either ShotIR field-level diffs or
 111 generator-agnostic repair plans. Each plan preserves the triggering code, evidence span, affected
 112 target, and rationale. In closed-loop evaluation, a video passes when all failure confidences fall
 113 below $\tau = 0.5$ or the loop reaches $K_{\max} = 3$; because this automatic criterion is verifier-dependent,
 114 Section 4.5 also reports blind human repair validation.

115 4 Experiments

116 4.1 Benchmark and Evaluation Protocol

117 We evaluate VideoGenDoctor as a diagnosis-and-repair system. The main questions are whether
 118 predicted failure codes are correct, evidence spans are temporally localized, repair plans are valid and
 119 useful, and repaired videos satisfy the specification under independent human judgment.

120 **Controlled fixture.** VideoGenDoctor-Bench-v0 contains 324 perturbed videos, 2,016 verified
 121 evidence spans, 9 deterministic perturbation types, and 12 observed failure codes. It is built from 18
 122 clean source clips across six domain groups; each clip is paired with a ShotIR-style specification and
 123 perturbed by nine deterministic operators under two seeds. The fixture validates interface consistency,
 124 evidence auditability, and repair-planning usefulness under controlled failures; it is not intended as a
 125 comprehensive open-world benchmark. Construction details and operator-to-template alignment are
 126 moved to Appendix A, Table 11, and Appendix B, Table 14.

Real-failure and real-normal subsets. We collect 240 naturally occurring failure videos from CogVideoX, Wan, Stable Video Diffusion, and HunyuanVideo Yang et al. [2024], Wan-AI [2025], Blattmann et al. [2023], Tencent Hunyuan Foundation Model Team [2025]. The subset is failure-enriched: from 480 raw generations, we retain videos only when annotators verify at least one visible specification violation. Sampling is balanced across the four generators at 60 retained failure videos each, with generator-specific input adapters logged in a per-video JSON manifest: CogVideoX and Wan use ShotIR-to-prompt text templates, Stable Video Diffusion uses reference-frame conditioning, and HunyuanVideo uses a multi-shot prompt template. The manifest records prompt or spec identifiers, converted inputs, generation settings, screening status, verified codes, and evidence spans so that the transfer check remains auditable even though it is not a prevalence estimate. We also retain 120 real-normal videos from the same raw generations to estimate false positives and unnecessary repair behavior. Ambiguous and low-quality exclusions are treated as stress analyses rather than standard accuracy benchmarks. Detailed source composition, normal-set behavior, temporal evidence profiles, and stress tables are reported in Appendix B.

Annotation. A reliability subset of 120 videos and 768 candidate evidence spans is dual-annotated before adjudication. We report code-level Cohen’s κ and span-level annotator-to-annotator tIoU in Appendix B, Table 20. Agreement is substantial for failure codes and lower for exact temporal boundaries, reflecting the ambiguity of fine-grained evidence spans.

Table 2: Dataset statistics. The first row is the expanded controlled perturbation fixture. The second row is the expanded real-failure subset collected from naturally occurring failures across multiple generators.

Split	#Vid	#Evidence	AvgDur	PerturbTypes	#Codes
Controlled fixture	324	2016	10.46s	9	12
Real-failure subset	240	1536	8.93s	–	18

4.2 Compared Methods and Metrics

We compare internal diagnosis variants (Prior-only, Rule-only, Rule+DummyJudge, Rule+Open-VLM, Rule+GPT-4V, and VideoGenDoctor-diagnosis), repair variants (VideoGenDoctor-patch, Patch+Judge, and VideoGenDoctor-full), and external VLM baselines including free-form VLM reporting, structured VLM reporting, self-consistency, VLM temporal proposal + patch, VLM-to-patch, and Score+LLM-to-patch. Most direct VLM baselines share the same hosted GPT-4V endpoint as Rule+GPT-4V to isolate prompt structure, output schema, and repair-planning protocol rather than model backbone. Whenever a table reports human repair success for a diagnosis-oriented front end, that front end is paired with the same downstream repair loop so that only the diagnostic interface changes. Implementation details are reported in Appendix B, Table 21; strengthened VLM-agent results are summarized in Appendix A, Table 12.

We report macro-F1 and micro-F1 for failure-code detection, tIoU@0.3/0.5 and Top- K keyframe hit rate for localization, automatic and human Pass@1/2 for repair, patch usefulness, new-artifact rate, and video-level bootstrap 95% confidence intervals. All span-aware metrics use the same matching protocol: records are grouped by failure code, target-incompatible predictions are filtered, and remaining pairs are matched by Hungarian assignment Kuhn [1955] using tIoU as the primary score and confidence only as a tie-breaker. Invalid or missing spans receive no localization credit.

4.3 Failure-Code Detection

Table 3: Failure-code detection on the expanded controlled fixture. Direct VLM baselines are included to compare VideoGenDoctor with strong agent-style alternatives. Confidence intervals are estimated by video-level bootstrap resampling. Best values are boldfaced.

Method	Macro-F1	Micro-F1	95% CI
Prior-only	0.4479	0.4635	[0.419, 0.477]
Rule+DummyJudge	0.8584	0.8716	[0.836, 0.879]
Rule-only	0.8926	0.9038	[0.872, 0.911]
Rule+Open-VLM	0.9047	0.9175	[0.885, 0.922]
VideoGenDoctor-diagnosis	0.9110	0.9243	[0.893, 0.928]
Rule+GPT-4V	0.9276	0.9386	[0.912, 0.941]
VLM free-form report	0.8402	0.8627	[0.814, 0.866]
VLM structured report	0.8869	0.9018	[0.864, 0.908]

VideoGenDoctor-diagnosis outperforms free-form and structured direct VLM reporting on macro-F1, while Rule+GPT-4V remains the strongest configuration. The larger gains in the localization results below indicate that the main advantage of structured diagnosis is not only detecting that a failure exists, but grounding it stably enough to support repair.

4.4 Per-Code Reliability and Evidence Localization

The controlled fixture covers twelve observed codes. Detailed per-code precision, recall, F1, and tIoU are reported in Appendix B, Table 22; the confusion matrix below summarizes dominant error modes.

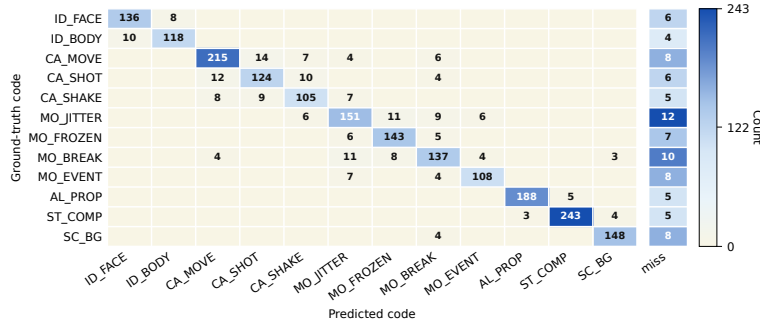


Figure 2: Per-code confusion matrix for the 12 observed taxonomy codes on the expanded controlled fixture. The strongest off-diagonal confusions occur among camera and motion failures, while identity, prop-missing, compression-artifact, and scene-background failures remain comparatively concentrated on the diagonal.

Table 4: Evidence localization against human evidence spans on the expanded controlled fixture. Confidence intervals are estimated by video-level bootstrap resampling. Best values are boldfaced.

Method	tIoU@0.3	tIoU@0.5	Top-1	Top-3	95% CI
Prior-only	0.5817	0.4295	0.1188	0.3315	[0.403, 0.457]
Rule+DummyJudge	0.7282	0.6128	0.2241	0.4259	[0.586, 0.639]
Rule-only	0.7836	0.6714	0.2670	0.5123	[0.645, 0.698]
Rule+Open-VLM	0.8089	0.7062	0.3565	0.5941	[0.681, 0.731]
VideoGenDoctor-diagnosis	0.8261	0.7316	0.4336	0.6815	[0.707, 0.754]
Rule+GPT-4V	0.8554	0.7688	0.4923	0.7747	[0.746, 0.790]
VLM free-form report	0.7204	0.5986	0.2994	0.5117	[0.570, 0.625]
VLM structured report	0.7812	0.6639	0.3651	0.5858	[0.637, 0.690]

Table 5: Closed-loop repair evaluation on the expanded controlled fixture. Automatic metrics are computed on all 324 controlled videos. Human metrics are computed on a stratified blind-evaluation subset of 144 videos. Best pass rates and patch-usefulness values are boldfaced; the new-artifact rate is reported for context rather than bold-ranked because abstention-heavy baselines are not directly comparable.

Method	Auto P@1	95% CI	Auto P@2	95% CI	Human Pass@1	95% CI	Human Pass@2	95% CI	Patch useful	New artifacts
Score-only	0.2407	[0.196, 0.291]	0.3580	[0.307, 0.411]	0.1875	[0.130, 0.257]	0.2778	[0.207, 0.355]	2.28	0.0833
Patch-only	0.5247	[0.471, 0.579]	0.6975	[0.646, 0.744]	0.4792	[0.399, 0.559]	0.6319	[0.551, 0.706]	3.50	0.1736
VLM-to-patch	0.5617	[0.507, 0.615]	0.7438	[0.694, 0.788]	0.5069	[0.427, 0.586]	0.6736	[0.594, 0.746]	3.64	0.2153
Patch+Judge	0.6296	[0.575, 0.681]	0.8179	[0.773, 0.857]	0.5625	[0.482, 0.640]	0.7431	[0.667, 0.807]	3.93	0.1458
VideoGenDoctor-full	0.6481	[0.594, 0.699]	0.8364	[0.793, 0.873]	0.5833	[0.503, 0.660]	0.7639	[0.689, 0.826]	4.11	0.1319

4.5 Closed-Loop Repair and Human Validation

Closed-loop repair is evaluated on the controlled fixture. Automatic Pass@1/2 is computed on all 324 videos; blind human Pass@1/2 is computed on a stratified 144-video subset. Human raters see the specification and output video but not method names or diagnosis records.

The main repair gain is most consistent with structured repair planning and verification rather than with the final loop wrapper alone. The paired comparison in Section 4.7 shows that the difference between Patch+Judge and VideoGenDoctor-full is small and not statistically decisive, so the full loop is best interpreted as traceable workflow integration.

4.6 Controlled Fixture versus Real Generator Failures

Table 6: Generalization check on expanded controlled perturbations and naturally occurring generator failures. Macro-F1 and tIoU@0.5 are computed from each method’s diagnosis front end; Patch useful and Human Pass@2 evaluate the corresponding downstream repair protocol. Best values within each subset are boldfaced.

Subset	Method	Macro-F1	tIoU@0.5	Patch useful	Human Pass@2
Controlled fixture	VLM structured report + repair loop	0.8869	0.6639	3.62	0.6810
Controlled fixture	VideoGenDoctor default pipeline	0.9110	0.7316	4.11	0.7639
Real-failure subset	VLM structured report + repair loop	0.7992	0.5431	3.36	0.6075
Real-failure subset	VideoGenDoctor default pipeline	0.8264	0.5982	3.82	0.6908

All methods degrade on naturally occurring failures, confirming that the controlled fixture is not a substitute for open-world evaluation. The real-failure subset contains more overlapping artifacts, less isolated evidence, and weaker alignment between specification fields and visible deviations. Per-code and per-generator results are reported in Appendix B, Tables 23 and 24.

4.7 Ablation and Generalization Checks

Table 7: Repair ablation on the expanded controlled fixture. We isolate the contribution of failure codes, temporal spans, evidence keyframes, patch templates, and verification. Best utility values are boldfaced; the new-artifact rate is reported for context rather than bold-ranked.

Repair setting	Code	Target	Span	Evidence	Template	Human Pass@2	Patch useful	New artifacts
Score-only	✗	✗	✗	✗	✗	0.2778	2.28	0.0833
Code-only patch	✓	✗	✗	✗	✓	0.4583	2.91	0.1597
Code+target patch	✓	✓	✗	✗	✓	0.5417	3.24	0.1736
Code+span patch	✓	✗	✓	✗	✓	0.6250	3.57	0.1667
Code+span+evidence patch	✓	✗	✓	✓	✓	0.6875	3.80	0.1528
VLM-to-patch	optional	optional	optional	optional	✗	0.6736	3.64	0.2153
Patch+Judge	✓	✓	✓	✓	✓	0.7431	3.93	0.1458
VideoGenDoctor-full	✓	✓	✓	✓	✓	0.7639	4.11	0.1319

The ablation tests whether localized evidence and template-constrained repair contribute beyond code prediction alone. The improvement from code-only to code+span and code+span+evidence settings supports the claim that evidence localization is operationally useful for repair planning.

Table 8: Paired comparison of repair variants. We report paired bootstrap differences in blind human Pass@2. Small differences between Patch+Judge and Full loop should be interpreted cautiously.

Comparison	Δ Human Pass@2	95% CI	Interpretation
Patch-only vs Score-only	0.3541	[0.250, 0.458]	substantial gain
VLM-to-patch vs Patch-only	0.0417	[-0.054, 0.139]	small / not decisive
Patch+Judge vs Patch-only	0.1112	[0.021, 0.215]	verification helps
VideoGenDoctor-full vs Patch+Judge	0.0208	[-0.063, 0.104]	small / not decisive

We also perform leave-one-perturbation-out and leave-one-source-group-out evaluations. Performance drops under held-out perturbations, especially for missing-event, camera-shake, and temporal-discontinuity operators, but remains above prior-only and score-only baselines. These results suggest that the system is not merely memorizing perturbation templates, while also confirming that unseen-operator generalization is weaker than in-distribution controlled evaluation. Detailed tables and the cost-performance trade-off are reported in Appendix B, Tables 25, 26, and Appendix A, Table 13.

5 Discussion

The current evidence supports a scoped claim: structured code–span–target–plan records improve diagnosis and repair planning over score-only feedback and direct VLM reporting on the controlled fixture and the current real-failure subset. It does not show that VideoGenDoctor solves open-world video repair. The controlled fixture demonstrates auditable localization under known perturbations, while the real-failure subset provides an initial but still limited transfer check.

The value of VideoGenDoctor should be assessed as an interface contribution rather than as a new video generation model or a learned repair algorithm. Its advantages lie in schema validity, temporal grounding, repair-plan usefulness, human auditability, and cost-performance. Rule+GPT-4V + repair loop is the strongest absolute scorer in the current controlled evaluation, but it relies on a more expensive hosted front end and is best interpreted as an upper-reference configuration rather than the default operating point. We therefore headline VideoGenDoctor-full because it preserves the structured interface and audit trail while delivering the main closed-loop gain at the paper’s default cost point. The modular design also allows feature extractors, rules, VLM judges, and patch compilers to be replaced independently; Stage-2 judging is restricted to top-ranked candidate segments, and the closed loop bounds repair by $K_{\max}$.

Limitations. The expanded fixture remains controlled; the real-failure subset is failure-enriched and cannot estimate natural failure prevalence; automatic pass rates are partly verifier-dependent; only observed taxonomy codes support empirical claims; and repair assumes access to a generator or editor that can consume patch actions. Broader validation requires larger generator coverage, longer videos, more domains and styles, stronger adapter executability tests, and larger multi-annotator studies.

6 Conclusion

We presented VideoGenDoctor, a structured framework for failure diagnosis, evidence localization, and repair planning in controllable video generation. The core contribution is an auditable code–span–target–plan interface that connects evaluation to actionable repair planning. The expanded evaluation consists of a 324-video controlled diagnostic fixture with 2,016 verified evidence spans and a 240-video real-failure subset with 1,536 verified evidence spans. The controlled fixture covers 12 observed taxonomy codes under 9 deterministic perturbation operators, while the real-failure subset covers 18 observed taxonomy codes across four publicly identifiable video generation systems. The primary empirical gain comes from structured repair planning and independent verification, while the full closed loop mainly provides traceable integration from diagnosis to evidence, patch planning, re-rendering, and verification. The broader generality claim remains scoped by the controlled and failure-enriched nature of the current evaluation sets.

7 Ethics Statement

VideoGenDoctor is a diagnostic tool intended to improve the quality and controllability of generated videos. Any tool that improves generation reliability could also be misused to strengthen misleading media. In sensitive deployments, automated repair should be paired with provenance tracking, watermarking, human review, and access control. Evidence localization may surface keyframes containing sensitive information, so downstream systems should support local processing, keyframe redaction, coarse time-binning, and privacy-preserving aggregation where appropriate. The taxonomy also encodes normative judgments about what counts as a failure; creative workflows should expose thresholds and permit human override.

8 Reproducibility and Release

For artifact review, the planned release package includes the reference implementation, report schema, taxonomy, patch compiler, controlled fixture generation scripts, annotations, prediction logs, evaluation scripts, prompt templates for VLM baselines, and scripts for bootstrap confidence intervals. Each run is expected to record configuration files, model versions, package versions, random seeds, prompts, raw VLM responses, and output paths. The intended artifact should permit CPU-only recomputation of tables from serialized predictions and annotations in seconds; full feature extraction and VLM judging remain configuration-dependent.

References

- Andreas Blattmann et al. Stable video diffusion: Scaling latent video diffusion models to large datasets, 2023. URL <https://arxiv.org/abs/2311.15127>. arXiv preprint arXiv:2311.15127.
- Gary Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000.
- Zhe Chen et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. *CVPR*, 2024.
- Mehdi Cherti et al. Reproducible scaling laws for contrastive language-image learning. *CVPR*, 2023.
- Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. *CVPR*, 2019.
- Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Scandinavian Conference on Image Analysis*, pages 363–370. Springer, 2003.
- Xuan He et al. VideoScore: Building automatic metrics to simulate fine-grained human feedback for video generation. *ACL*, 2024.
- Ziqi Huang, Yinan He, Jiashuo Yu, et al. VBench: Comprehensive benchmark suite for video generative models. *CVPR*, 2024.
- Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *NeurIPS*, 2023.
- Yaofang Liu et al. EvalCrafter: Benchmarking and evaluating large video generation models. *CVPR*, 2024.
- OpenAI. GPT-4V(ision) system card. Technical report, OpenAI, 2023. URL <https://openai.com/research/gpt-4v-system-card>.
- Kaiyue Sun, Kaiyi Huang, Xian Liu, Yue Wu, Zihan Xu, Zhenguo Li, and Xihui Liu. T2V-CompBench: A comprehensive benchmark for compositional text-to-video generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8406–8416, 2025.
- Zachary Teed and Jia Deng. RAFT: Recurrent all-pairs field transforms for optical flow. *ECCV*, 2020.
- Tencent Hunyuan Foundation Model Team. Hunyuanvideo 1.5 technical report, 2025. URL <https://arxiv.org/abs/2511.18870>.

- 271 Ultralytics. YOLOv8. Software repository, 2023. URL <https://github.com/ultralytics/ultralytics>.
272 Version 8.0.
- 273 Wan-AI. Wan2.1-t2v-14b. Hugging Face model card, 2025. URL <https://huggingface.co/Wan-AI/Wan2>
274 .1-T2V-14B.
- 275 Zhouxia Wang, Ziyang Yuan, Xintao Wang, Tianshui Chen, Menghan Xia, Ping Luo, and Ying Shan. MotionCtrl:
276 A unified and flexible motion controller for video generation, 2023. URL <https://arxiv.org/abs/2312>
277 .03641. arXiv preprint arXiv:2312.03641.
- 278 Zhuoyi Yang et al. CogVideoX: Text-to-video diffusion models with an expert transformer, 2024. URL
279 <https://arxiv.org/abs/2408.06072>. arXiv preprint arXiv:2408.06072.
- 280 Hangjie Yuan, Shiwei Zhang, Xiang Wang, Yujie Wei, Tao Feng, Yining Pan, Yingya Zhang, Ziwei Liu,
281 Samuel Albanie, and Dong Ni. InstructVideo: Instructing video diffusion models with human feedback.
282 In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages
283 6463–6474, 2024.

284 A Supplementary Main-Text Results

285 This appendix stores tables and figures moved out of the main text to keep the core narrative compact
286 while preserving the audit trail for comparisons, taxonomy scope, fixture construction, temporal
287 evidence profiles, strengthened VLM baselines, repair visualization, and cost-performance trade-offs.

Table 9: Capability comparison with related evaluation and critique paradigms. Here “Global score” denotes scalar quality, alignment, or verifier-style pass outputs rather than localized evidence records. Direct VLM reporting is treated as a strong baseline rather than only comparing against score-only benchmarks.

System	Global score	Code	Evidence	Patch	Closed-Loop
VBench	✓	✗	✗	✗	✗
EvalCrafter	✓	✗	✗	✗	✗
VideoScore	✓	✗	✗	✗	✗
T2V-CompBench	✓	partial	✗	✗	✗
Direct VLM report	partial	partial	partial	partial	✗
VideoGenDoctor	✓	✓	✓	✓	✓

Table 10: Failure-as-code taxonomy groups. Each group defines a namespace for diagnosis records and links to evidence procedures and repair-plan templates.

Group	Operational scope
Identity	Identity drift, face/body consistency, wrong character, clothing consistency.
Camera	Shot type, camera movement, viewpoint angle, unintended zoom, camera shake.
Motion	Action timing, frame continuity, motion physics, frozen frames, segment breaks.
Alignment	Prompt adherence, required objects and props, character count, prop placement.
Style	Color consistency, texture flicker, compression artifacts, art-style consistency.
Scene	Background consistency, lighting, environment match, scene-object stability.

Table 11: Composition of the expanded controlled diagnostic fixture. Each domain group contains three clean source clips. Every source clip is paired with a ShotIR-style specification and perturbed by nine deterministic operators under two random seeds.

Domain group	#Source clips	Avg. duration	Resolution	#Perturb. ops	#Seeds	#Videos
Indoor / human-object	3	10.1s	768×432	9	2	54
Outdoor / motion	3	10.8s	768×432	9	2	54
Multi-object interaction	3	10.4s	768×432	9	2	54
Camera-control scene	3	11.0s	768×432	9	2	54
Style / lighting variation	3	10.2s	768×432	9	2	54
Scene-change / background	3	10.3s	768×432	9	2	54
Total	18	10.46s	—	9	2	324

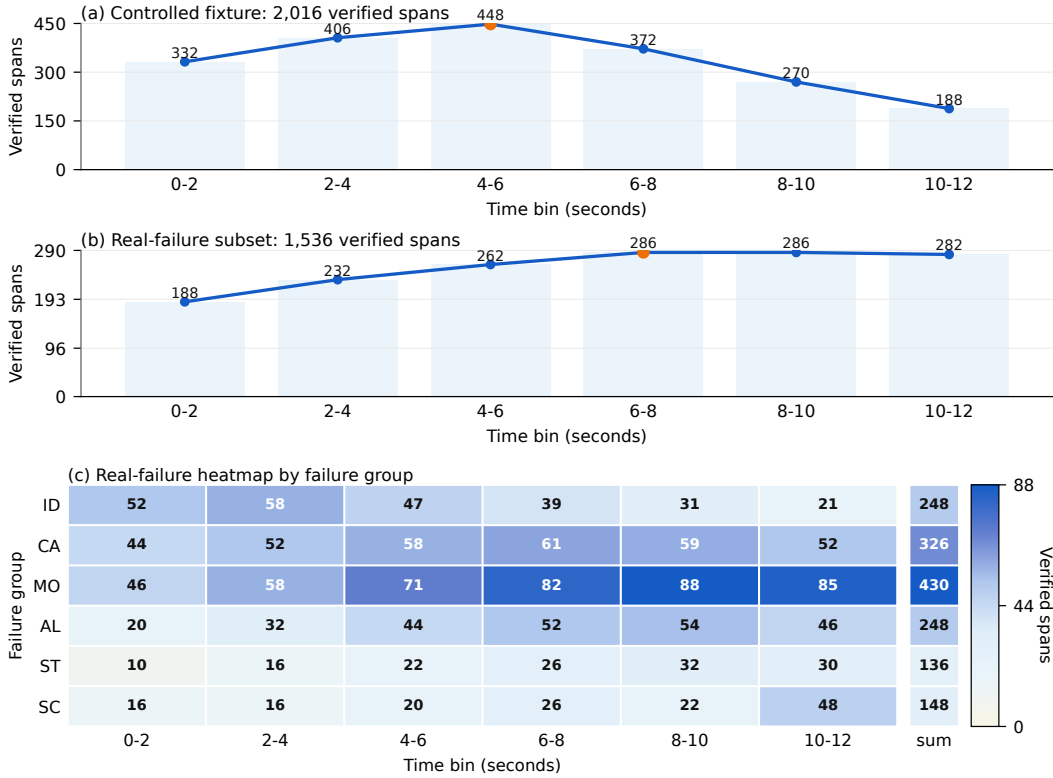


Figure 3: Temporal distribution of verified evidence spans in the expanded evaluation sets. Panel (a) aggregates the 2,016 controlled-fixture spans over 2-second bins and shows the front- and mid-loaded pattern expected from deterministic perturbation operators. Panel (b) summarizes the 1,536 real-failure spans over the same time bins, making the later and more diffuse temporal mass directly comparable to panel (a). Panel (c) further breaks the real-failure spans down by failure group and time bin, highlighting that motion and camera failures dominate the later intervals and overlap more strongly than in the controlled fixture.

Table 12: Strengthened VLM-agent baselines on the expanded controlled fixture. All rows are evaluated under the same downstream repair protocol; diagnosis-only front ends are explicitly paired with that shared repair loop when human repair success is reported. Self-consistency improves direct VLM reporting but increases cost. The validity column reports whether the emitted action stays inside the supported repair-plan vocabulary; it does not claim generator-adaptor executability. Best values are boldfaced; higher is better except for relative cost, where lower is better.

Method	Macro-F1	tIoU@0.5	Human Pass@2	Schema validity	Patch vocabulary validity	Relative cost
VLM free-form report	0.8402	0.5986	0.6125	0.742	0.604	1.12×
VLM structured report	0.8869	0.6639	0.6810	0.914	0.732	1.16×
VLM structured + self-consistency	0.8978	0.6824	0.7042	0.951	0.755	3.48×
VLM temporal proposal + patch	0.8915	0.7018	0.7076	0.936	0.802	2.05×
VideoGenDoctor-full	0.9110	0.7316	0.7639	0.991	0.953	1.00×
Rule+GPT-4V + repair loop	0.9276	0.7688	0.7917	0.993	0.957	2.75×

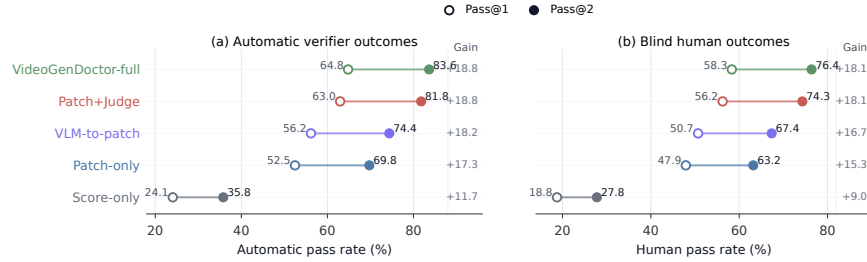


Figure 4: Closed-loop repair comparison on the expanded controlled fixture. Left: automatic verifier Pass@1/2 on all 324 controlled videos. Right: blind human Pass@1/2 on the stratified 144-video human-evaluation subset. Automatic pass rates may be coupled to the system verifier, while human pass rates provide an independent check of repair success.

Table 13: Cost-performance trade-off under a shared downstream repair protocol. Diagnosis-only front ends are paired with the same repair loop before reporting Human Pass@2. Cost is reported relative to the default VideoGenDoctor-full configuration. Values in the last two columns are multiplicative factors relative to VideoGenDoctor-full (1.00×). Best values are boldfaced; higher is better for task metrics, lower is better for cost and latency.

Method	Macro-F1	tIoU@0.5	Human Pass@2	Relative cost	Relative latency
Rule-only + repair loop	0.8926	0.6714	0.5988	0.42×	0.58×
Rule+Open-VLM + repair loop	0.9047	0.7062	0.7160	0.73×	0.87×
VideoGenDoctor-full	0.9110	0.7316	0.7639	1.00×	1.00×
Rule+GPT-4V + repair loop	0.9276	0.7688	0.7917	2.75×	2.31×
VLM structured report	0.8869	0.6639	0.6810	1.16×	1.20×

288 B Additional Experimental Tables

289 This appendix collects construction details, audit tables, stress analyses, and per-slice results that
290 support the main experimental claims without interrupting the main narrative.

Table 14: Operator-template alignment analysis for the expanded controlled fixture. The table makes explicit which observable cues, rule signals, failure codes, and repair-plan templates are aligned by construction.

Perturbation operator	Observable cue	Rule / feature signal	Failure code	Repair-plan template	Alignment type
Identity-anchor removal	face or body appearance drift	face embedding drift; character-region embedding drift	ID_FACE_DRIFT, ID_BODY_DRIFT	inject_reference_frame; replace_character_anchor	identity-anchor
Required-prop dropping	required object becomes absent or inconsistent	object detector absence; VLM prop-absence judgment	AL_PROP_MISSING	inject_prop	prop-presence
Camera-movement alteration	requested trajectory is replaced by static or wrong motion	global flow direction; trajectory mismatch	CA_MOVE_WRONG	set_camera_movement	motion-control
Shot-type / framing alteration	framing violates requested shot type	crop statistics; VLM framing judgment	CA_SHOT_TYPE_WRONG	adjust_framing	framing-control
Camera-shake injection	unstable camera with unintended jitter	flow instability; camera-shake heuristic	CA_SHAKE	stabilize_camera	stability-control
Action dropping / shifting	required event missing or misordered	action-track gap; VLM event-order judgment	MO_EVENT_MISSING	inject_action_keyframe; regenerate_segment	event-structure
Temporal jitter / frame-drop injection	discontinuity, duplicate frames, or rate mismatch	flow anomaly; near-zero motion plateau	MO_JITTER, MO_SEGMENT_BREAK	normalize_frame_rate; regenerate_segment	temporal-continuity
Compression re-encoding	blockiness and de-blocking artifacts	compression detector; texture degradation signal	ST_COMPRESSION_ARTIFACT	apply_enhancement	quality-restoration
Background-anchor weakening	scene background drifts across the shot	scene embedding drift; background-anchor mismatch	SC_BG_INCONSISTENCY	fix_background_anchor	scene-anchor

Table 15: Source composition of the expanded real-failure subset. The subset is failure-enriched: we generate 480 raw videos and retain 240 samples with annotator-verified visible failures. Sampling is balanced at 60 retained failures per generator, and generator-specific prompt or conditioning adapters are logged in the construction manifest. The subset is used only as an initial transfer check, not to estimate natural failure prevalence. We use publicly identifiable generator checkpoints to make the subset construction auditable and reproducible.

Generator	Version / checkpoint	Input type	#Prompts / specs	#Videos	Avg. duration	#Unique observed codes
CogVideoX	CogVideoX1.5-5B	text / ShotIR-to-prompt	60	60	8.4s	15
Wan	Wan2.1-T2V-14B	text / ShotIR-to-prompt	60	60	9.2s	16
Stable Video Diffusion	SVD-XT-1.1	image-to-video / reference frame	60	60	8.6s	14
HunyuanVideo	HunyuanVideo-1.5	text / multi-shot prompt	60	60	9.5s	17
Total	–	–	240	240	8.93s	18

Table 16: Real-failure and real-normal evaluation. The real-normal subset contains 120 videos retained from the same 480 raw generations but judged to contain no obvious failure. Whenever a diagnosis-oriented front end is listed, patch-related columns are computed after pairing that front end with the shared downstream repair loop. Best values are boldfaced; higher is better except for real-normal FPR and unnecessary patch rate, where lower is better.

Method	Real-failure Recall	Real-normal FPR	Unnecessary Patch Rate	No-op Accuracy	Specificity
Score-only	0.621	0.258	0.231	0.769	0.742
Rule-only	0.806	0.168	0.154	0.846	0.832
VLM structured report	0.818	0.217	0.205	0.795	0.783
VLM structured + self-consistency	0.837	0.167	0.150	0.850	0.833
VLM temporal proposal + patch	0.848	0.192	0.183	0.817	0.808
VideoGenDoctor-diagnosis	0.862	0.108	0.092	0.908	0.892
VideoGenDoctor-full	0.858	0.100	0.087	0.913	0.900

Table 17: Stress analysis on 72 ambiguous specification-violation cases. These samples contain possible but non-adjudicable deviations from the specification. Whenever a diagnosis-oriented front end is listed, patch-related columns are computed after pairing that front end with the shared downstream repair loop. Best values are boldfaced; higher is better for abstention-oriented columns, lower is better for false concrete patch, wrong-target patch, over-repair, and new artifacts.

Method	Abstention rate	Appropriate abstention	False concrete patch	Wrong-target patch	Over-repair	New artifacts
Score-only	0.292	0.361	0.514	0.222	0.347	0.083
VLM structured report	0.181	0.292	0.611	0.278	0.431	0.153
VLM temporal proposal + patch	0.222	0.347	0.556	0.236	0.389	0.194
VideoGenDoctor-diagnosis	0.500	0.639	0.292	0.125	0.208	0.069
VideoGenDoctor-full	0.472	0.625	0.250	0.111	0.181	0.111

Table 18: Stress analysis on 48 low-quality or non-adjudicable cases. These samples are dominated by global degradation, unclear content, or insufficient evidence for reliable code-level labels. Whenever a diagnosis-oriented front end is listed, patch-related columns are computed after pairing that front end with the shared downstream repair loop. Best values are boldfaced; higher is better for quality-gated abstention, lower is better for the remaining columns.

Method	Quality-gated abstention	False concrete patch	Wrong-target patch	Over-repair	New artifacts
Score-only	0.417	0.375	0.125	0.208	0.063
VLM structured report	0.292	0.458	0.188	0.271	0.104
VLM temporal proposal + patch	0.333	0.396	0.167	0.250	0.146
VideoGenDoctor-diagnosis	0.667	0.146	0.063	0.104	0.042
VideoGenDoctor-full	0.625	0.125	0.042	0.083	0.083

Table 19: Observed failure-code distribution in the expanded controlled fixture and real-failure subset. The full taxonomy contains 38 codes, but empirical claims are restricted to the observed codes below.

Failure code	Group	Controlled spans	Real-failure spans	Total spans
ID_FACE_DRIFT	Identity	150	112	262
ID_BODY_DRIFT	Identity	132	76	208
ID_CLOTHING_CHANGE	Identity	–	60	60
CA_MOVE_WRONG	Camera	240	104	344
CA_SHOT_TYPE_WRONG	Camera	144	84	228
CA_SHAKE	Camera	126	70	196
CA_WRONG_ANGLE	Camera	–	68	68
MO_JITTER	Motion	174	124	298
MO_FROZEN_FRAME	Motion	156	72	228
MO_SEGMENT_BREAK	Motion	162	88	250
MO_EVENT_MISSING	Motion	126	90	216
MO_ACTION_ORDER_WRONG	Motion	–	56	56
AL_PROP_MISSING	Alignment	198	116	314
AL_TEXT_PROMPT_MISMATCH	Alignment	–	132	132
ST_COMPRESSION_ARTIFACT	Style	252	80	332
ST_COLOR_SHIFT	Style	–	56	56
SC_BG_INCONSISTENCY	Scene	156	86	242
SC_OBJECT_TELEPORT	Scene	–	62	62
Total	–	2016	1536	3552

Table 20: Annotation reliability. Dual annotation is performed on a reliability subset before adjudication.

Subset	#Videos	#Candidate spans	#Annotators	Code agreement	Span agreement
Controlled reliability subset	72	468	2	$\kappa = 0.858$	tIoU = 0.716
Real-failure reliability subset	48	300	2	$\kappa = 0.803$	tIoU = 0.641
Combined	120	768	2	$\kappa = 0.831$	tIoU = 0.681

Table 21: Implementation details for VLM-based judges and external diagnosis/repair baselines.

Setting	Model	Frames	Res.	Prompt	Rel. cost
Rule+Open-VLM	Qwen2-VL-7B-Instruct	18	512px short side	structured QA	$0.34\times$
Rule+GPT-4V	GPT-4V endpoint	18	512px short side	structured QA	$2.75\times$
VLM free-form report	GPT-4V endpoint	12	512px short side	free-form critique	$1.12\times$
VLM structured report	GPT-4V endpoint	12	512px short side	JSON schema	$1.16\times$
VLM structured + self-consistency	GPT-4V endpoint	12 frames $\times$ 3 runs	512px short side	JSON schema, 3 runs	$3.48\times$
VLM temporal proposal + patch	GPT-4V endpoint	12	512px short side	span proposal + JSON	$2.05\times$
VLM-to-patch	GPT-4V endpoint	12	512px short side	JSON patch	$1.19\times$
Score+LLM-to-patch	scalar evaluator + LLM	0	–	score-to-patch JSON	N/A

Note. Most direct VLM baselines share the same hosted GPT-4V endpoint OpenAI [2023] to isolate prompt- and schema-level effects rather than backbone changes; Rule+Open-VLM provides an open-model reference. Hosted endpoint calls are logged with raw responses for auditability. ‘N/A’ denotes a scalar-evaluator-dominated pipeline whose cost is not directly comparable to frame-based VLM calls.

Table 22: Per-code reliability of the default VideoGenDoctor diagnosis configuration on the expanded controlled fixture.

Observed code	Precision	Recall	F1	tIoU@0.5
ID_FACE_DRIFT	0.938	0.914	0.926	0.748
ID_BODY_DRIFT	0.922	0.897	0.909	0.725
CA_MOVE_WRONG	0.932	0.916	0.924	0.734
CA_SHOT_TYPE_WRONG	0.913	0.889	0.901	0.705
CA_SHAKE	0.901	0.878	0.889	0.692
MO_JITTER	0.890	0.865	0.877	0.676
MO_FROZEN_FRAME	0.963	0.950	0.956	0.794
MO_SEGMENT_BREAK	0.906	0.883	0.894	0.704
MO_EVENT_MISSING	0.875	0.848	0.861	0.662
AL_PROP_MISSING	0.951	0.930	0.940	0.760
ST_COMPRESSION_ARTIFACT	0.950	0.966	0.958	0.779
SC_BG_INCONSISTENCY	0.910	0.885	0.897	0.686

Table 23: Per-code reliability on the expanded real-failure subset. The subset is more challenging than the controlled fixture, especially for motion, camera, and scene-level failures.

Failure code	#Spans	Precision	Recall	F1	tIoU@0.5
ID_FACE_DRIFT	112	0.884	0.848	0.866	0.648
ID_BODY_DRIFT	76	0.856	0.823	0.839	0.615
ID_CLOTHING_CHANGE	60	0.834	0.798	0.816	0.586
CA_MOVE_WRONG	104	0.866	0.832	0.849	0.628
CA_SHOT_TYPE_WRONG	84	0.838	0.803	0.820	0.590
CA_SHAKE	70	0.820	0.779	0.799	0.559
CA_WRONG_ANGLE	68	0.825	0.786	0.805	0.568
MO_JITTER	124	0.823	0.777	0.799	0.565
MO_FROZEN_FRAME	72	0.899	0.854	0.876	0.663
MO_SEGMENT_BREAK	88	0.827	0.792	0.809	0.582
MO_EVENT_MISSING	90	0.807	0.762	0.784	0.542
MO_ACTION_ORDER_WRONG	56	0.792	0.748	0.769	0.524
AL_PROP_MISSING	116	0.893	0.863	0.878	0.669
AL_TEXT_PROMPT_MISMATCH	132	0.842	0.794	0.817	0.596
ST_COMPRESSION_ARTIFACT	80	0.914	0.874	0.893	0.688
ST_COLOR_SHIFT	56	0.846	0.806	0.825	0.601
SC_BG_INCONSISTENCY	86	0.832	0.795	0.813	0.578
SC_OBJECT_TELEPORT	62	0.823	0.782	0.802	0.545

Table 24: Per-generator results on the real-failure and real-normal subsets. FPR is computed on 30 real-normal samples per generator.

Generator	Macro-F1	tIoU@0.5	Human Pass@2	Real-normal FPR	Main failure groups	Normal samples
CogVideoX	0.834	0.609	0.702	0.092	Camera, Motion, Alignment	30
Wan	0.821	0.588	0.681	0.108	Motion, Camera, Identity	30
Stable Video Diffusion	0.842	0.617	0.696	0.083	Identity, Style, Scene	30
HunyuanVideo	0.809	0.579	0.684	0.117	Camera, Motion, Alignment	30
Average	0.827	0.598	0.691	0.100	–	120

Table 25: Leave-one-perturbation-out evaluation. Each held-out operator is excluded from threshold tuning and then used only for testing.

Held-out perturbation operator	Macro-F1	tIoU@0.5	Top-3 hit
Identity-anchor removal	0.884	0.681	0.644
Required-prop dropping	0.902	0.704	0.672
Camera-movement alteration	0.871	0.653	0.612
Shot-type / framing alteration	0.862	0.641	0.606
Camera-shake injection	0.851	0.628	0.590
Action dropping / shifting	0.838	0.612	0.568
Temporal jitter / frame-drop injection	0.846	0.621	0.577
Compression re-encoding	0.910	0.719	0.688
Background-anchor weakening	0.868	0.646	0.610
Average	0.870	0.656	0.619

Table 26: Leave-one-source-group-out evaluation over the six controlled-fixture domain groups.

Held-out source group	Macro-F1	tIoU@0.5	Top-3 hit
Indoor / human-object	0.899	0.714	0.665
Outdoor / motion	0.887	0.702	0.651
Multi-object interaction	0.892	0.706	0.657
Camera-control scene	0.875	0.683	0.631
Style / lighting variation	0.907	0.722	0.676
Scene-change / background	0.884	0.694	0.642
Average	0.891	0.704	0.654

291 C Full Failure Taxonomy

292 Table 27 reports the machine-readable taxonomy used by VideoGenDoctor v0.1. The v0.1 taxonomy
 293 contains 38 codes grouped into six categories. Each code is paired with an operational definition,
 294 an evidence procedure, and a default patch template. The current benchmark evaluates the 12 codes
 295 observed in VideoGenDoctor-Bench-v0, while the remaining codes define the extensible namespace
 296 used by the report schema and patch compiler.

Table 27: Full VideoGenDoctor v0.1 failure taxonomy used by the report schema and patch compiler.

Code	Definition	Evidence Procedure	Patch Template
Identity			
ID_FACE_DRIFT	Face identity changes inconsistently between frames or segments.	Compare face embeddings across keyframes and flag large cosine distance.	inject_ reference_ frame
ID_BODY_DRIFT	Body appearance, clothing, or build changes inconsistently.	Compare character-region embeddings across segments.	inject_ reference_ frame
ID_WRONG_CHARACTER	A visible character does not match any specified character.	Check face embeddings against known character anchors.	replace_ character_ anchor

Continued on next page

Code	Definition	Evidence Procedure	Patch Template
ID_CLOTHING_CHANGE	Clothing changes between shots without narrative justification.	Measure character-region appearance drift across segments.	fix_character_consistency
ID_MULTIPLE_FACES	Multiple distinct faces appear when one character is expected.	Count distinct identities detected in a single-character shot.	reduce_character_count
ID_NO_FACE_VISIBLE	An expected face is absent when the shot requires face visibility.	Check face detection under close-up or medium-shot constraints.	adjust_framing
Scene			
SC_BG_INCONSISTENCY	The background changes unexpectedly between segments.	Measure background-region embedding drift across consecutive segments.	fix_background_anchor
SC_ENVIRONMENT_MISMATCH	The scene environment does not match the specified location.	Compare frames with the location description using image-text similarity.	set_environment
SC_LIGHTING_SHIFT	Lighting changes inconsistently within a shot.	Measure per-frame luminance variation within the segment.	normalize_lighting
SC_OBJECT_TELEPORT	A scene object appears or disappears without plausible motion.	Track object boxes across adjacent frames.	stabilize_scene_objects
SC_BG_FLICKER	The background flickers or pulses unnaturally.	Detect high-frequency luminance variation in background regions.	smooth_background
SC_WRONG_TIME_OF_DAY	The time of day does not match the specification.	Classify sky and ambient lighting against the expected time.	set_time_of_day
Motion			
MO_JITTER	Unnatural frame-to-frame jitter appears outside intended camera motion.	Measure optical-flow magnitude variance and direction instability.	normalize_frame_rate
MO_FRAME_DROP	Duplicate or dropped frames produce visible stutter.	Detect near-zero adjacent-frame flow when motion is expected.	interpolate_frames
MO_UNNATURAL_MOTION	Character or object motion is physically implausible.	Compare flow direction and magnitude with expected motion constraints.	constrain_motion
MO_EVENT_MISSING	A required action or event does not occur in the expected segment.	Ask the VLM judge whether the specified action is visible.	inject_action_keyframe
MO_ACTION_ORDER_WRONG	Actions occur in the wrong temporal order.	Compare detected action timestamps with the specification order.	reorder_segments
MO_MOTION_BLUR_EXCESS	Excessive motion blur degrades keyframes.	Use sharpness measures such as Laplacian variance.	reduce_motion_blur
MO_FROZEN_FRAME	The video is frozen when motion is expected.	Detect near-zero flow across the full segment.	regenerate_segment
MO_SEGMENT_BREAK	A visual discontinuity occurs at a segment boundary.	Detect embedding-drift spikes at boundaries.	smooth_segment_transition
Camera			
CA_MOVE_WRONG	Camera movement deviates from the specified movement.	Compare optical-flow direction patterns with the expected movement type.	set_camera_movement
CA_SHOT_TYPE_WRONG	The shot type does not match the specification.	Compare face or body box size ratios with the expected shot type.	adjust_framing
CA_UNINTENDED_ZOOM	Unplanned zoom-in or zoom-out occurs.	Detect radial flow without a zoom specification.	remove_zoom
CA_SHAKE	Camera shake appears outside the specified movement.	Detect high-frequency oscillation in global optical flow.	stabilize_camera
CA_WRONG_ANGLE	Camera angle deviates from the specification.	Ask the VLM judge to compare the observed angle with the target angle.	set_camera_angle
CA_ROLL	Unintended camera roll appears.	Estimate the rotational component of optical flow.	remove_camera_roll
Alignment			
AL_PROP_MISSING	A required prop is not visible in the segment.	Use object detection or VLM verification for prop absence.	inject_prop
AL_PROP_WRONG_PLACEMENT	A required prop appears in the wrong region.	Compare the prop box with the expected screen region.	reposition_prop

Continued on next page

Code	Definition	Evidence Procedure	Patch Template
AL_TEXT_PROMPT_MISMATCH	The generated content does not align with the prompt.	Measure text-image similarity and verify mismatches with a judge.	strengthen_prompt_guidance
AL_CHARACTER_COUNT_WRONG	The number of visible characters does not match the specification.	Count faces or bodies and compare with the expected count.	adjust_character_count
AL_WRONG_PROP	A visible prop is not listed in the specification.	Detect high-confidence unlisted objects.	remove_prop
AL_PROP_OCCLUDED	A required prop is present but substantially occluded.	Estimate visible area and occlusion of the prop box.	adjust_prop_placement
Style			
ST_COLOR_SHIFT	The color palette changes inconsistently across segments.	Measure HSV histogram distance between segments.	apply_color_grading
ST_ART_STYLE_INCONSISTENCY	The art style shifts between segments.	Compare style embeddings across segments.	apply_style_transfer
ST_COMPRESSION_ARTIFACT	Visible compression artifacts degrade video quality.	Detect blocking, ringing, or quality-score degradation.	apply_enhancement
ST_TEXTURE_FLICKER	Texture patterns flicker or shimmer unnaturally.	Measure high temporal frequency in texture regions.	smooth_texture
ST_OVEREXPOSURE	Significant frame regions are overexposed.	Count the fraction of saturated high-value pixels.	adjust_exposure
ST_UNDEREXPOSURE	Significant frame regions are underexposed.	Count the fraction of near-black pixels.	adjust_exposure

297 D Judge Protocol

298 The Stage-2 judge receives the top- K candidate segments produced by Stage-1. For each segment, the
299 input includes the segment identifier, temporal bounds, keyframe paths, candidate failure codes, and
300 specification context, including required props, expected actions, camera movement, shot type, and
301 character anchors. The judge answers structured questions rather than producing a free-form report.
302 This design keeps the output comparable across local open-source VLMs and hosted vision-capable
303 models.

304 For identity failures, the judge checks whether the face, body appearance, clothing, or other distinctive
305 visual attributes of the main character change inconsistently across keyframes. For alignment failures,
306 it verifies whether required props are visible, whether they appear in the expected screen region, and
307 whether the generated content matches the prompt or structured specification. For camera failures, it
308 compares the observed camera movement, shot type, and viewpoint angle with the requested control
309 signal. For motion failures, it verifies whether required actions occur, whether they occur in the
310 specified order, and whether stutter, frozen frames, or segment breaks are visible. For style and scene
311 failures, it checks lighting, color, compression artifacts, background consistency, and environment
312 match.

313 The output is a structured object containing a natural-language answer for each question, a confidence
314 score, per-code probability updates, and a re-ranking of keyframes. The final failure score is computed
315 as

$$\tilde{\sigma}_{s,c} = (1 - \alpha)\sigma_{s,c}^{(1)} + \alpha\sigma_{s,c}^{(2)},$$

316 where $\sigma_{s,c}^{(1)}$ is the Stage-1 score, $\sigma_{s,c}^{(2)}$ is the judge score, and $\alpha = 0.6$ by default. All judge calls
317 record the model name, prompt, raw response, token count when available, and output path to support
318 reproducibility.

319 E Patch Template Examples

320 VideoGenDoctor compiles each diagnosis record into one or more patch actions. If a ShotIR
321 specification is available, the compiler emits field-level diffs. If the generator does not expose a
322 structured specification interface, the compiler emits generator-agnostic repair plans that can be
323 translated into prompt edits, reference-frame injection, segment-level re-rendering, or post-processing
324 steps.

```
325 {"action": "inject_reference_frame",
326   "field": "character_id",
```

```

327 "value": "<anchor_frame>",
328 "reason": "ID_FACE_DRIFT detected at t=1.2s"}

329 {"action": "set_camera_movement",
330 "field": "camera_movement",
331 "value": "<expected_camera_movement>",
332 "reason": "CA_MOVE_WRONG detected in the localized span"}

333 {"action": "inject_prop",
334 "field": "props_required",
335 "value": "<missing_prop>",
336 "reason": "AL_PROP_MISSING verified by object or VLM evidence"}

337 {"action": "regenerate_segment",
338 "field": "segment",
339 "value": "<t0,t1>",
340 "reason": "MO_FROZEN_FRAME or MO_SEGMENT_BREAK is localized to this span"}

341 Each patch specifies an action type, the affected target, an optional update value, and a short rationale
342 tied to the evidence span. When multiple records affect the same target, the compiler groups them
343 before re-rendering to reduce conflicts among patch actions.

```

Table 28: Patch-to-generator adapter details. A repair plan is counted as adapter-executable only when it can be automatically translated into target-generator or editor inputs without manual rewriting. L0/L1 rows denote abstention or prompt-rewrite instructions rather than executable patches.

Repair-plan action	ShotIR field	Counted as	Adapter availability	Concrete generator/editor input	Fallback
inject_reference_frame	character.identity_anchor	L2	L2 available for SVD; L1 partial for text-to-video models	reference image input + identity prompt strengthening	prompt-only repair plan
set_camera_movement	camera.movement	L1	L1 partial	camera-control prompt rewrite; trajectory adapter when supported	no_op_uncertain if no camera control exists
set_camera_angle	camera.viewpoint	L1	L1 partial	viewpoint prompt rewrite; optional camera-control token	repair suggestion only
inject_prop	props.required	L1 / L2	L1 available; L2 partial with mask/reference input	prop phrase insertion + optional reference or mask	prompt edit
reposition_prop	props.region	L1	L1 partial	region-aware prompt rewrite or mask-conditioned edit	prompt-only repair plan
regenerate_segment	temporal span	L2 / L3	L2 available when segment rerendering is supported	segment-level rerender or temporal inpainting call	full-clip rerender
stabilize_camera	camera.constraint	L1 / L2	L1 partial	no-shake constraint or stabilization post-process	repair suggestion only
normalize_frame_rate	motion.continuity	L2	L2 partial	frame interpolation or temporal smoothing post-process	regenerate_segment
apply_enhancement	style.quality	L2	L2 available	enhancement / deblocking post-process	prompt edit for higher quality
fix_background_anchor	scene.background_anchor	L1	L1 partial	background-anchor prompt strengthening; optional reference image	prompt-only repair plan
no_op_uncertain	–	L0	L0 available	abstain from concrete repair	no action

344 In the reported Human Pass@K evaluation, the concretely executed actions are the L2/L3 subset
345 exposed by the active backend, primarily reference-frame injection, segment rerendering or temporal
346 smoothing, enhancement or deblocking, and mask- or reference-conditioned prop insertion when
347 supported. L0/L1 rows remain structured repair-plan instructions and are not counted as executable
348 patches.

349 F Annotation Guide Summary

350 Annotators review each perturbed video together with the original prompt or ShotIR specification
351 and the auto-assigned candidate failure codes. For each candidate, annotators determine whether
352 the failure is visually present, assign the tightest temporal span that contains the visible deviation,

and select representative keyframes when available. Annotators may also add missing failures if the perturbation produces an additional visible artifact.

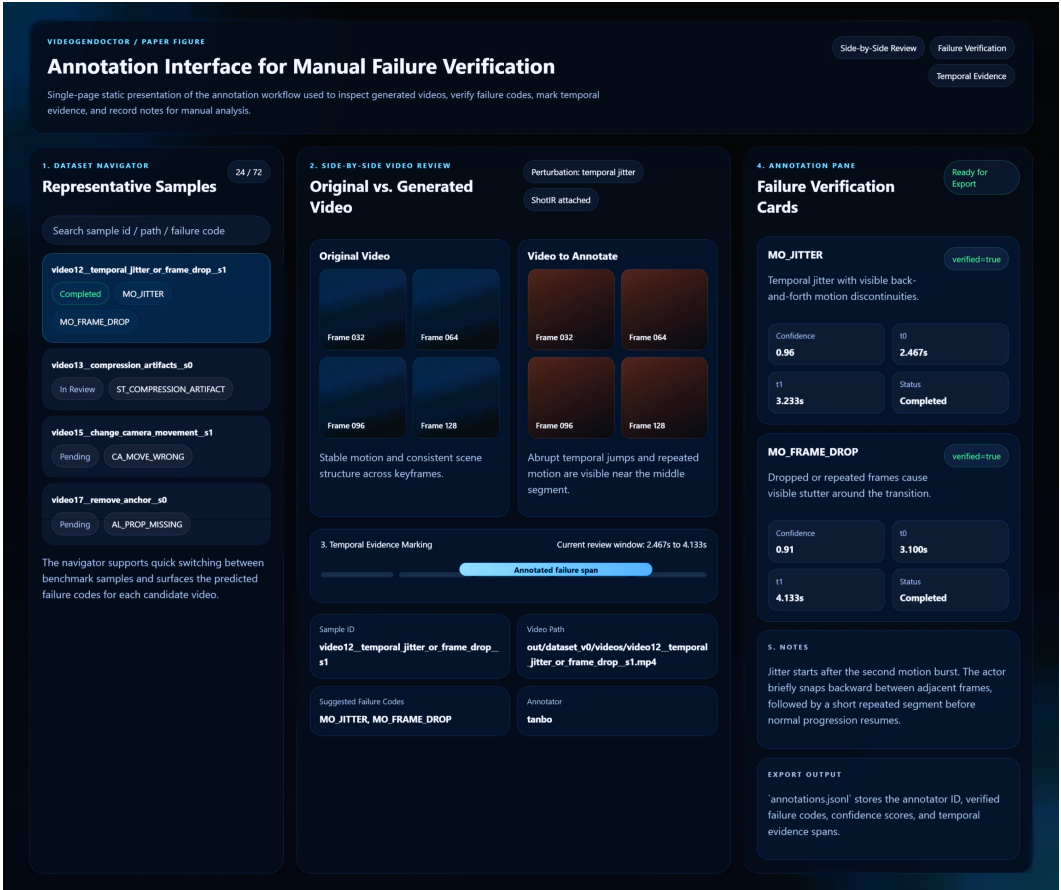


Figure 5: Annotation interface used for manual verification in VideoGenDoctor-Bench-v0. The tool exposes a sample navigator, side-by-side review of the original and perturbed videos, temporal evidence marking, structured failure verification cards, and free-form notes for adjudication.

The annotation record is stored as one JSON object per video and includes the video identifier, annotator identifier, verified failure codes, confidence values, evidence spans, and free-form notes. A failure is retained in the benchmark only when the annotator can verify it from the rendered video and specification, without relying on perturbation metadata alone. Ambiguous cases are handled conservatively: if a deviation is not visually identifiable, the candidate is marked as not verified.

The expanded experiment fixture includes a dual-annotated reliability subset of 120 videos and 768 candidate evidence spans. The subset contains both controlled perturbation samples and naturally occurring real-failure samples. Larger multi-annotator validation with more than two independent annotators remains future work.

364 **G Example Diagnostic Report**

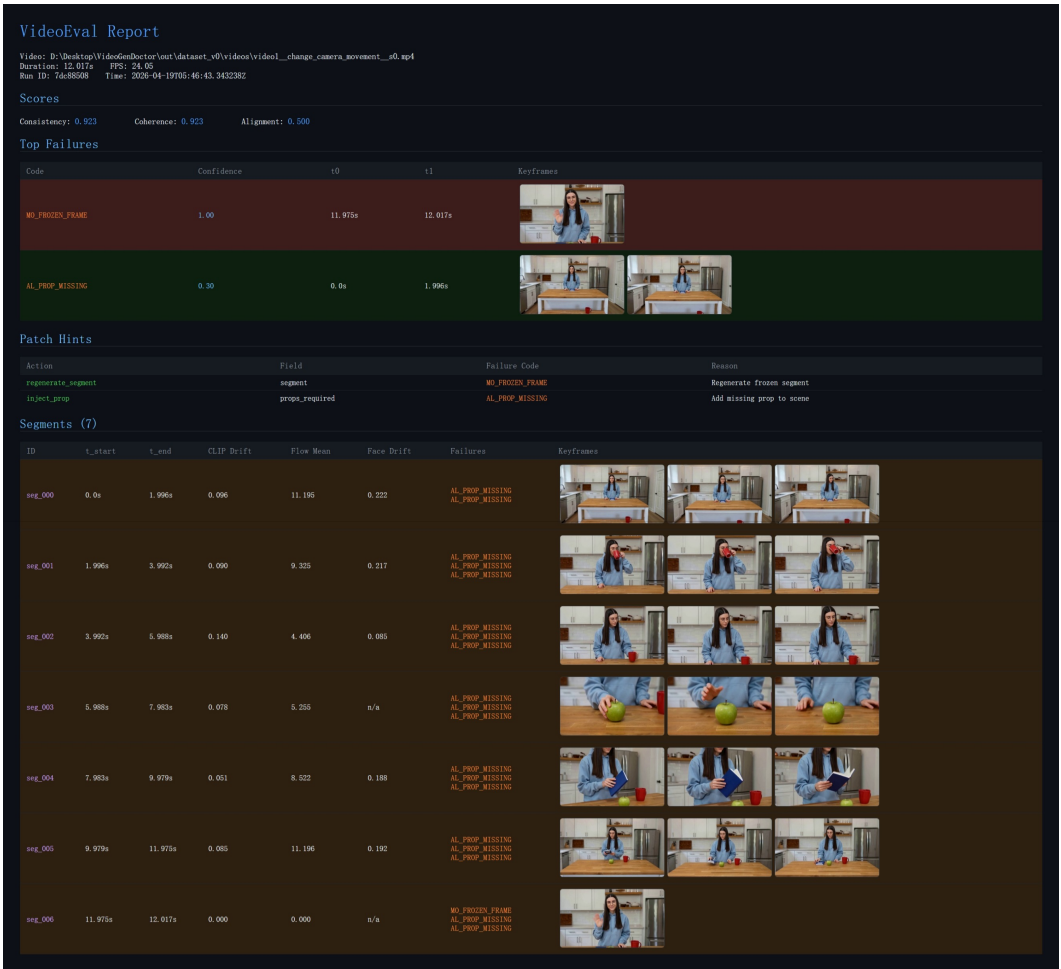


Figure 6: Example VideoGenDoctor diagnostic report for a perturbed video. The report summarizes global scores, ranked failure records with confidence and temporal spans, representative keyframes, patch hints derived from the retained failure codes, and segment-level evidence used for localized review and repair.

365 **H Direct VLM Baseline Prompt Templates**

366 The four direct VLM/LLM baselines differ only in their allowed information access and output schema.
367 The free-form and structured VLM report baselines receive sampled frames and the specification.
368 The VLM-to-patch baseline also receives sampled frames, the taxonomy names, and the repair-plan
369 vocabulary, but not VideoGenDoctor’s rule scores or patch compiler outputs. The Score+LLM-to-
370 patch baseline receives scalar evaluator scores and the specification only; it does not receive sampled
371 frames. These information boundaries are fixed before evaluation and are used for all samples.

372 **F.1 Free-form VLM report prompt.** The model receives the video specification, sampled
373 keyframes with timestamps, and a short instruction: identify visible failures, explain the evidence,
374 and propose repair suggestions. The response is then parsed into taxonomy codes and evidence spans
375 for evaluation.

376 **F.2 Structured VLM report prompt.** The model receives the same input but must output
377 JSON objects with fields failure_code, temporal_span, affected_target, confidence,
378 evidence_keyframes, rationale, and repair_action. Failures without visual evidence must
379 be marked as uncertain rather than hallucinated.

```

380 You are evaluating whether a generated video follows the given specification.
381 For each visible failure, return a JSON record:
382 {
383     "failure_code": "<taxonomy code>",
384     "temporal_span": [t0, t1],
385     "affected_target": "<character / prop / camera / motion / style / scene>",
386     "confidence": <0-1>,
387     "evidence_keyframes": ["frame@time", ...],
388     "rationale": "short visual reason",
389     "repair_action": "<patch action>"
390 }
391 Only report failures that are visually supported by the frames.

```

F.3 VLM-to-patch prompt. The VLM-to-patch baseline receives the same video specification and sampled keyframes as the direct VLM diagnosis baselines, but it is asked to output repair actions directly rather than diagnosis records. This baseline is allowed to access the taxonomy names and the supported repair-plan vocabulary, but it is not given VideoGenDoctor’s Stage-1 rule scores, candidate segments, or patch compiler outputs. It may infer temporal spans from the sampled frames, but it does not receive ground-truth spans or VideoGenDoctor-predicted spans. This setting tests whether a VLM can directly produce structured repair plans from the specification and visual evidence.

```

399 You are a video-generation repair planner.
400
401 Input:
402 1. A video specification or ShotIR-style instruction.
403 2. Sampled keyframes from the generated video, each with a timestamp.
404 3. The failure-code taxonomy names.
405 4. The supported repair-plan vocabulary.
406
407 Your task:
408 Inspect the sampled frames and the specification. Identify only visually supported
409 problems and output structured repair-plan actions. Do not write a free-form critique.
410 Do not invent failures that are not visible in the frames.
411
412 You may use the following failure-code groups:
413 - Identity: ID_FACE_DRIFT, ID_BODY_DRIFT, ID_WRONG_CHARACTER,
414   ID_CLOTHING_CHANGE, ID_MULTIPLE_FACES, ID_NO_FACE_VISIBLE
415 - Camera: CA_MOVE_WRONG, CA_SHOT_TYPE_WRONG, CA_UNINTENDED_ZOOM,
416   CA_SHAKE, CA_WRONG_ANGLE, CA_ROLL
417 - Motion: MO_JITTER, MO_FRAME_DROP, MO_UNNATURAL_MOTION,
418   MO_EVENT_MISSING, MO_ACTION_ORDER_WRONG, MO_MOTION_BLUR_EXCESS,
419   MO_FROZEN_FRAME, MO_SEGMENT_BREAK
420 - Alignment: AL_PROP_MISSING, AL_PROP_WRONG_PLACEMENT,
421   AL_TEXT_PROMPT_MISMATCH, AL_CHARACTER_COUNT_WRONG,
422   AL_WRONG_PROP, AL_PROP_OCCLUDED
423 - Style: ST_COLOR_SHIFT, ST_ART_STYLE_INCONSISTENCY,
424   ST_COMPRESSION_ARTIFACT, ST_TEXTURE_FLICKER,
425   ST_OVEREXPOSURE, ST_UNDEREXPOSURE
426 - Scene: SC_BG_INCONSISTENCY, SC_ENVIRONMENT_MISMATCH,
427   SC_LIGHTING_SHIFT, SC_OBJECT_TELEPORT, SC_BG_FLICKER,
428   SC_WRONG_TIME_OF_DAY
429
430 Supported repair-plan vocabulary:
431 - inject_reference_frame
432 - replace_character_anchor
433 - fix_character_consistency
434 - adjust_framing
435 - set_camera_movement
436 - set_camera_angle

```

```

437 - stabilize_camera
438 - remove_zoom
439 - remove_camera_roll
440 - inject_prop
441 - reposition_prop
442 - remove_prop
443 - adjust_character_count
444 - strengthen_prompt_guidance
445 - inject_action_keyframe
446 - reorder_segments
447 - regenerate_segment
448 - interpolate_frames
449 - normalize_frame_rate
450 - constrain_motion
451 - smooth_segment_transition
452 - reduce_motion_blur
453 - apply_color_grading
454 - apply_style_transfer
455 - apply_enhancement
456 - smooth_texture
457 - adjust_exposure
458 - fix_background_anchor
459 - set_environment
460 - normalize_lighting
461 - stabilize_scene_objects
462 - set_time_of_day
463 - no_op_uncertain
464
465 Return a JSON object with the following schema:
466 {
467   "patches": [
468     {
469       "patch_id": "<unique patch id>",
470       "failure_code": "<taxonomy code or uncertain>",
471       "action": "<one item from the supported repair-plan vocabulary>",
472       "target": "<character / prop / camera / motion / style / scene target>",
473       "temporal_span": [<start_sec>, <end_sec>],
474       "evidence_keyframes": ["frame@time", "..."],
475       "confidence": <0-1>,
476       "patch_fields": {
477         "field": "<generator or ShotIR field to modify>",
478         "value": "<new value, constraint, or reference to insert>"
479       },
480       "rationale": "<short visual reason>"
481     }
482   ],
483   "global_notes": "<one-sentence summary>",
484   "uncertain": <true or false>
485 }
486
487 Rules:
488 1. Output valid JSON only.
489 2. Every patch must use an action from the supported repair-plan vocabulary.
490 3. If no visually supported failure is visible, return:
491   {"patches": [], "global_notes": "No visually supported repair action.", "uncertain": true}
492 4. If the temporal span is uncertain, estimate it from the nearest sampled frame
493    timestamps and set "confidence" below 0.5.
494 5. Do not output patches for purely aesthetic preferences unless the specification
495    explicitly requires them.

```

F4 Score+LLM-to-patch prompt. The Score+LLM-to-patch baseline receives scalar evaluator outputs and the original video specification, but it does not receive sampled frames. This baseline is intentionally weaker in visual grounding but represents a common production setting where low-level evaluators return scalar scores and an LLM converts low-scoring dimensions into repair suggestions. Because it cannot inspect frames, it is not allowed to claim frame-specific visual evidence. It may output approximate temporal spans only when the scalar evaluator provides segment-level scores; otherwise the temporal span must be marked as null. This makes the information boundary explicit and prevents unfair comparison with frame-based VLM baselines.

You are a repair planner for controllable video generation.

Input:

1. The original video specification or ShotIR-style instruction.
2. Scalar evaluator outputs, such as quality, alignment, motion, identity, camera, style, and scene-consistency scores.
3. Optional segment-level scalar scores if available.
4. The supported repair-plan vocabulary.

Important information boundary:

You do NOT receive sampled frames or the video itself. Therefore:

- Do not claim that a failure is visually observed.
- Do not mention specific visual evidence or keyframes.
- Only infer likely repair actions from low-scoring evaluator dimensions.
- Use "evidence_source": "scalar_score" for all patches.
- Use "temporal_span": null unless segment-level scores identify a specific interval.

Supported repair-plan vocabulary:

- inject_reference_frame
- replace_character_anchor
- fix_character_consistency
- adjust_framing
- set_camera_movement
- set_camera_angle
- stabilize_camera
- remove_zoom
- remove_camera_roll
- inject_prop
- reposition_prop
- remove_prop
- adjust_character_count
- strengthen_prompt_guidance
- inject_action_keyframe
- reorder_segments
- regenerate_segment
- interpolate_frames
- normalize_frame_rate
- constrain_motion
- smooth_segment_transition
- reduce_motion_blur
- apply_color_grading
- apply_style_transfer
- apply_enhancement
- smooth_texture
- adjust_exposure
- fix_background_anchor
- set_environment
- normalize_lighting
- stabilize_scene_objects
- set_time_of_day


```

554 - no_op_uncertain
555
556 Return a JSON object with the following schema:
557 {
558   "patches": [
559     {
560       "patch_id": "<unique patch id>",
561       "score_dimension": "<low-scoring evaluator dimension>",
562       "inferred_failure_type": "<short inferred failure description>",
563       "action": "<one item from the supported repair-plan vocabulary>",
564       "target": "<character / prop / camera / motion / style / scene target>",
565       "temporal_span": null,
566       "evidence_source": "scalar_score",
567       "triggering_scores": {
568         "<score_name>": <score_value>
569       },
570       "confidence": <0-1>,
571       "patch_fields": {
572         "field": "<generator or ShotIR field to modify>",
573         "value": "<new value, constraint, or reference to insert>"
574       },
575       "rationale": "<short reason based only on scalar scores and specification>"
576     }
577   ],
578   "global_notes": "<one-sentence summary>",
579   "uncertain": <true or false>
580 }
581
582 Rules:
583 1. Output valid JSON only.
584 2. Do not claim visual evidence because frames are not provided.
585 3. Do not output frame IDs or evidence keyframes.
586 4. Use temporal_span = null unless segment-level scalar scores are provided.
587 5. Every patch must use an action from the supported repair-plan vocabulary.
588 6. If the scalar scores do not indicate a clear repair target, return:
589   {"patches": [], "global_notes": "No reliable score-based repair action.", "uncertain": true}

```

590 **F.5 Parsing and invalid-output handling.** Invalid JSON outputs are repaired using a deterministic
591 parser when possible. If the output cannot be parsed, the prediction is marked invalid. Failure
592 codes outside the taxonomy are mapped to the nearest taxonomy code only when an exact synonym
593 exists; otherwise they are counted as false positives. Predictions without temporal spans receive
594 no localization credit. Hallucinated failures without visual evidence are counted as false positives.
595 Repair actions outside the supported repair-plan vocabulary are marked invalid for repair-plan scoring
596 and non-adapter-executable for adapter scoring.

597 I Blind Human Repair Evaluation Protocol

598 Raters are shown the specification and one output video, without method names or diagnosis records.
599 They answer whether the video satisfies the specification overall, whether the target failure is resolved,
600 whether the repair introduces new artifacts, and how useful the patch is on a 1–5 scale. Human
601 Pass@ K is the fraction of samples judged successful after at most K repair attempts.

Table 29: Blind human repair evaluation rubric for patch usefulness.

Score	Criterion
1	The patch does not address the target failure.
2	The patch is related to the failure but is not actionable or is largely insufficient.
3	The patch is actionable but incomplete or requires substantial manual interpretation.
4	The patch directly addresses the target failure with minor ambiguity.
5	The patch directly resolves the target failure and preserves surrounding valid content.

A repaired video is marked as pass if the target failure is resolved and no severe new artifact is introduced. Human Pass@K is the fraction of samples judged successful after at most K repair attempts. New artifacts are marked when raters observe a visible degradation introduced by repair that was not present in the original generated video. Each repaired video is rated by 3 independent raters. We report the majority-vote pass/fail label, the mean patch-usefulness score, and the majority-vote new-artifact label.

Table 30: Error analysis on ambiguous and real-normal samples. The table focuses on no-op behavior and wrong-target repair planning. Best values are boldfaced for interpretable comparison columns; the no_op_uncertain rate is reported descriptively rather than bold-ranked.

Method	no_op_uncertain rate	Correct no-op rate	Missed necessary repair	Wrong-target patch	Over-confident patch
Score-only	0.286	0.524	0.119	0.171	0.321
VLM structured report	0.219	0.476	0.096	0.229	0.396
VLM temporal proposal + patch	0.257	0.518	0.108	0.200	0.350
VideoGenDoctor-diagnosis	0.552	0.738	0.142	0.095	0.181
VideoGenDoctor-full	0.514	0.724	0.156	0.081	0.164

Table 31: Qualitative case-study summary. Cases are sampled from controlled, real-failure, and ambiguous stress subsets.

Case type	#Cases	Majority-judged successful / appropriate	Typical outcome
Successful structured repair plan	24	18	localized repair plan resolves target failure
Ambiguous case with abstention	24	16	no_op_uncertain avoids over-repair
Over-repair / wrong-target failure	24	7	concrete patch targets wrong object or span

J Bootstrap Confidence Intervals

We use 10,000 video-level bootstrap resamples and report percentile 95% confidence intervals. For the controlled fixture, resampling is performed over 324 videos. For the real-failure subset, resampling is performed over 240 videos. For blind human repair evaluation, resampling is performed over the stratified human-rated subset. For paired comparisons, we resample videos with replacement and compute the distribution of metric differences between two methods. Small point-estimate differences are not treated as meaningful unless the paired confidence interval excludes zero.

Table 32: Leave-one-code-family-out rule ablation. For the held-out family, dedicated family-specific rules are disabled; only generic embedding drift, optical-flow anomaly, scene-change peaks, and structured VLM verification are retained.

Held-out family	Macro-F1	tIoU@0.5	Top-3 hit	Main degradation source
Identity	0.782	0.579	0.536	face/body-specific anchors removed
Camera	0.754	0.546	0.501	camera-flow templates disabled
Motion	0.721	0.512	0.468	jitter / segment-break rules disabled
Alignment	0.801	0.603	0.558	prop detector and placement rules disabled
Style	0.836	0.641	0.590	compression/style detectors disabled
Scene	0.748	0.528	0.487	background-anchor rules disabled
Average	0.774	0.568	0.523	–

The large degradation under leave-one-code-family-out ablation confirms that VideoGenDoctor should not be interpreted as an unconstrained open-ended failure discovery system. The experiment clarifies which portions of the performance depend on family-specific detectors and which portions remain supported by generic evidence signals and VLM verification.

K Real-Failure Construction Manifest

Table 33: Generator configuration metadata for the real-failure and real-normal subsets. Hosted or version-changing endpoints are logged with access date and endpoint identifier.

Generator	Checkpoint / endpoint	fps	Frames	Resolution	Steps	Guidance	Scheduler	Seed range	Reference source / prompt rule
CogVideoX	CogVideoX1.5-5B	16	128	768×432	50	6.0	DPMSolver	10000–10059	ShotIR-to-prompt template v1
Wan	Wan2.1-T2V-14B	16	128	768×432	50	5.0	FlowMatchEuler	18000–18059	ShotIR-to-prompt template v1
Stable Video Diffusion	SVD-XT-1.1	14	112	768×432	40	7.5	EulerDiscrete	26000–26059	reference frame from ShotIR anchor
HunyuanVideo	HunyuanVideo-1.5	16	128	768×432	50	7.0	FlowMatchEuler	34000–34059	multi-shot prompt template v1

When a generator does not expose one of these configuration fields, the corresponding manifest value is set to null rather than omitted. For hosted or version-changing endpoints, we log the endpoint identifier, access date, raw request payload, and raw response metadata.

To make the real-failure subset auditable and reproducible, each generated video is accompanied by a JSON manifest. The manifest records the generator identity, checkpoint, input specification, generation configuration, output location, screening decision, verified failure codes, temporal evidence spans, and annotator identifiers. This protocol turns the real-failure subset from an informal collection of generated examples into a traceable evaluation resource.

Each manifest entry contains the following fields: `generator`, `checkpoint`, `prompt_id`, `input_type`, `shotir_spec`, `shotir_to_prompt_converted_text`, `reference_frame_id`, `seed`, `resolution`, `fps`, `num_frames`, `sampling_steps`, `guidance_scale`, `output_path`, `screening_status`, `verified_codes`, `evidence_spans`, and `annotator_ids`. The `screening_status` field takes one of four values: `generated`, `retained_candidate`, `verified`, or `excluded_ambiguous`. Only entries marked as `verified` are included in the reported real-failure evaluation.

```
{
  "video_id": "real_wan_00037",
  "generator": "Wan",
  "checkpoint": "Wan2.1-T2V-14B",
  "prompt_id": "shotir_prompt_037",
  "input_type": "text / ShotIR-to-prompt",
  "shotir_spec": {
    "shot_id": "S037",
    "duration_sec": 8.0,
    "characters": [
      {
        "id": "person_A",
        "description": "young man wearing a blue hoodie",
        "identity_anchor": "ref_person_A_004"
      }
    ],
    "required_props": [
      {
        "id": "red_water_bottle",
        "description": "red water bottle held in the right hand",
        "expected_presence": "visible throughout the shot"
      }
    ],
    "camera": {
      "shot_type": "medium shot",
      "movement": "slow left-to-right tracking",

```

```

661     "viewpoint": "front three-quarter view"
662 },
663     "action_order": [
664         "person_A enters the park path",
665         "person_A jogs while holding the red water bottle",
666         "person_A slows down near the bench"
667     ],
668     "style": {
669         "lighting": "natural daylight",
670         "visual_style": "realistic video"
671     },
672     "scene": {
673         "location": "urban park path",
674         "background_anchor": "trees and bench remain stable"
675     }
676 },
677     "shotir_to_prompt_converted_text": "A realistic 8-second medium shot of a young man wearing a k
678     "reference_frame_id": null,
679     "seed": 18473,
680     "resolution": "768x432",
681     "fps": 16,
682     "num_frames": 128,
683     "sampling_steps": 50,
684     "guidance_scale": 7.5,
685     "output_path": "real_failures/wan/real_wan_00037.mp4",
686     "screening_status": "verified",
687     "verified_codes": [
688         "AL_PROP_MISSING",
689         "CA_MOVE_WRONG",
690         "MO_SEGMENT_BREAK"
691     ],
692     "evidence_spans": [
693         {
694             "failure_code": "AL_PROP_MISSING",
695             "target": "red_water_bottle",
696             "start_sec": 3.25,
697             "end_sec": 6.75,
698             "evidence_keyframes": [
699                 "frame_052@3.25s",
700                 "frame_080@5.00s",
701                 "frame_108@6.75s"
702             ],
703             "annotator_confidence": 0.91,
704             "notes": "The specified red water bottle is not visible after the character begins jogging
705         },
706         {
707             "failure_code": "CA_MOVE_WRONG",
708             "target": "camera_movement",
709             "start_sec": 0.00,
710             "end_sec": 4.00,
711             "evidence_keyframes": [
712                 "frame_000@0.00s",
713                 "frame_032@2.00s",
714                 "frame_064@4.00s"
715             ],
716             "annotator_confidence": 0.84,
717             "notes": "The camera remains mostly static rather than performing the requested left-to-right
718         },
719         {

```

```

720     "failure_code": "MO_SEGMENT_BREAK",
721     "target": "temporal_continuity",
722     "start_sec": 5.50,
723     "end_sec": 5.94,
724     "evidence_keyframes": [
725         "frame_088@5.50s",
726         "frame_095@5.94s"
727     ],
728     "annotator_confidence": 0.78,
729     "notes": "A visible discontinuity occurs near the transition to the final action."
730 }
731 ],
732 "annotator_ids": [
733     "ann_02",
734     "ann_05"
735 ]
736 }

```

737 For privacy and release safety, file paths may be anonymized, but the manifest must preserve all fields
738 required to reproduce the generation configuration, reconstruct the ShotIR-to-prompt conversion, and
739 audit the verified evidence spans. When a generator does not expose a configuration field such as
740 guidance_scale, the field is set to null rather than omitted.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: **[TODO]**

788 Justification: **[TODO]**

789 Guidelines:

- 790 • The answer **[N/A]** means that the paper has no limitation while the answer **[No]** means that
- 791 the paper has limitations, but those are not discussed in the paper.
- 792 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 793 • The paper should point out any strong assumptions and how robust the results are to vi-
- 794 olations of these assumptions (e.g., independence assumptions, noiseless settings, model
- 795 well-specification, asymptotic approximations only holding locally). The authors should
- 796 reflect on how these assumptions might be violated in practice and what the implications
- 797 would be.
- 798 • The authors should reflect on the scope of the claims made, e.g., if the approach was only
- 799 tested on a few datasets or with a few runs. In general, empirical results often depend on
- 800 implicit assumptions, which should be articulated.
- 801 • The authors should reflect on the factors that influence the performance of the approach. For
- 802 example, a facial recognition algorithm may perform poorly when image resolution is low or
- 803 images are taken in low lighting. Or a speech-to-text system might not be used reliably to
- 804 provide closed captions for online lectures because it fails to handle technical jargon.
- 805 • The authors should discuss the computational efficiency of the proposed algorithms and how
- 806 they scale with dataset size.
- 807 • If applicable, the authors should discuss possible limitations of their approach to address
- 808 problems of privacy and fairness.
- 809 • While the authors might fear that complete honesty about limitations might be used by review-
- 810 ers as grounds for rejection, a worse outcome might be that reviewers discover limitations that
- 811 aren’t acknowledged in the paper. The authors should use their best judgment and recognize
- 812 that individual actions in favor of transparency play an important role in developing norms
- 813 that preserve the integrity of the community. Reviewers will be specifically instructed to not
- 814 penalize honesty concerning limitations.

815 **3. Theory assumptions and proofs**

816 Question: For each theoretical result, does the paper provide the full set of assumptions and a

817 complete (and correct) proof?

818 Answer: **[TODO]**

819 Justification: **[TODO]**

820 Guidelines:

- 821 • The answer **[N/A]** means that the paper does not include theoretical results.
- 822 • All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- 823 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 824 • The proofs can either appear in the main paper or the supplemental material, but if they appear
- 825 in the supplemental material, the authors are encouraged to provide a short proof sketch to
- 826 provide intuition.
- 827 • Inversely, any informal proof provided in the core of the paper should be complemented by
- 828 formal proofs provided in appendix or supplemental material.
- 829 • Theorems and Lemmas that the proof relies upon should be properly referenced.

830 **4. Experimental result reproducibility**

831 Question: Does the paper fully disclose all the information needed to reproduce the main

832 experimental results of the paper to the extent that it affects the main claims and/or conclusions

833 of the paper (regardless of whether the code and data are provided or not)?

834 Answer: **[TODO]**

835 Justification: **[TODO]**

836 Guidelines:

- 837 • The answer **[N/A]** means that the paper does not include experiments.

- 838 • If the paper includes experiments, a [No] answer to this question will not be perceived well
839 by the reviewers: Making the paper reproducible is important, regardless of whether the code
840 and data are provided or not.
- 841 • If the contribution is a dataset and/or model, the authors should describe the steps taken to
842 make their results reproducible or verifiable.
- 843 • Depending on the contribution, reproducibility can be accomplished in various ways. For
844 example, if the contribution is a novel architecture, describing the architecture fully might
845 suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary
846 to either make it possible for others to replicate the model with the same dataset, or provide
847 access to the model. In general, releasing code and data is often one good way to accomplish
848 this, but reproducibility can also be provided via detailed instructions for how to replicate the
849 results, access to a hosted model (e.g., in the case of a large language model), releasing of a
850 model checkpoint, or other means that are appropriate to the research performed.
- 851 • While NeurIPS does not require releasing code, the conference does require all submissions
852 to provide some reasonable avenue for reproducibility, which may depend on the nature of
853 the contribution. For example
 - 854 (a) If the contribution is primarily a new algorithm, the paper should make it clear how to
855 reproduce that algorithm.
 - 856 (b) If the contribution is primarily a new model architecture, the paper should describe the
857 architecture clearly and fully.
 - 858 (c) If the contribution is a new model (e.g., a large language model), then there should either
859 be a way to access this model for reproducing the results or a way to reproduce the model
860 (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - 861 (d) We recognize that reproducibility may be tricky in some cases, in which case authors are
862 welcome to describe the particular way they provide for reproducibility. In the case of
863 closed-source models, it may be that access to the model is limited in some way (e.g.,
864 to registered users), but it should be possible for other researchers to have some path to
865 reproducing or verifying the results.

866 5. Open access to data and code

867 Question: Does the paper provide open access to the data and code, with sufficient instructions to
868 faithfully reproduce the main experimental results, as described in supplemental material?

869 Answer: [TODO]

870 Justification: [TODO]

871 Guidelines:

- 872 • The answer [N/A] means that paper does not include experiments requiring code.
- 873 • Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 874 • While we encourage the release of code and data, we understand that this might not be
875 possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including
876 code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- 877 • The instructions should contain the exact command and environment needed to run to
878 reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 879 • The authors should provide instructions on data access and preparation, including how to
880 access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 881 • The authors should provide scripts to reproduce all experimental results for the new proposed
882 method and baselines. If only a subset of experiments are reproducible, they should state
883 which ones are omitted from the script and why.
- 884 • At submission time, to preserve anonymity, the authors should release anonymized versions
885 (if applicable).
- 886 • Providing as much information as possible in supplemental material (appended to the paper)
887 is recommended, but including URLs to data and code is permitted.

890 **6. Experimental setting/details**

891 Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters,
892 how they were chosen, type of optimizer) necessary to understand the results?

893 Answer: **[TODO]**

894 Justification: **[TODO]**

895 Guidelines:

- 896 • The answer **[N/A]** means that the paper does not include experiments.
- 897 • The experimental setting should be presented in the core of the paper to a level of detail that
898 is necessary to appreciate the results and make sense of them.
- 899 • The full details can be provided either with the code, in appendix, or as supplemental material.

900 **7. Experiment statistical significance**

901 Question: Does the paper report error bars suitably and correctly defined or other appropriate
902 information about the statistical significance of the experiments?

903 Answer: **[TODO]**

904 Justification: **[TODO]**

905 Guidelines:

- 906 • The answer **[N/A]** means that the paper does not include experiments.
- 907 • The authors should answer **[Yes]** if the results are accompanied by error bars, confidence
908 intervals, or statistical significance tests, at least for the experiments that support the main
909 claims of the paper.
- 910 • The factors of variability that the error bars are capturing should be clearly stated (for example,
911 train/test split, initialization, random drawing of some parameter, or overall run with given
912 experimental conditions).
- 913 • The method for calculating the error bars should be explained (closed form formula, call to a
914 library function, bootstrap, etc.)
- 915 • The assumptions made should be given (e.g., Normally distributed errors).
- 916 • It should be clear whether the error bar is the standard deviation or the standard error of the
917 mean.
- 918 • It is OK to report 1-sigma error bars, but one should state it. The authors should preferably
919 report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality
920 of errors is not verified.
- 921 • For asymmetric distributions, the authors should be careful not to show in tables or figures
922 symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- 923 • If error bars are reported in tables or plots, the authors should explain in the text how they
924 were calculated and reference the corresponding figures or tables in the text.

925 **8. Experiments compute resources**

926 Question: For each experiment, does the paper provide sufficient information on the computer
927 resources (type of compute workers, memory, time of execution) needed to reproduce the
928 experiments?

929 Answer: **[TODO]**

930 Justification: **[TODO]**

931 Guidelines:

- 932 • The answer **[N/A]** means that the paper does not include experiments.
- 933 • The paper should indicate the type of compute workers CPU or GPU, internal cluster, or
934 cloud provider, including relevant memory and storage.
- 935 • The paper should provide the amount of compute required for each of the individual experi-
936 mental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer **[No]**, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that there is no societal impact of the work performed.
- If the authors answer **[N/A]** or **[No]**, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper poses no such risks.

987 • Released models that have a high risk for misuse or dual-use should be released with necessary
988 safeguards to allow for controlled use of the model, for example by requiring that users adhere
989 to usage guidelines or restrictions to access the model or implementing safety filters.

990 • Datasets that have been scraped from the Internet could pose safety risks. The authors should
991 describe how they avoided releasing unsafe images.

992 • We recognize that providing effective safeguards is challenging, and many papers do not
993 require this, but we encourage authors to take this into account and make a best faith effort.

994 **12. Licenses for existing assets**

995 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the
996 paper, properly credited and are the license and terms of use explicitly mentioned and properly
997 respected?

998 Answer: **[TODO]**

999 Justification: **[TODO]**

1000 Guidelines:

1001 • The answer **[N/A]** means that the paper does not use existing assets.

1002 • The authors should cite the original paper that produced the code package or dataset.

1003 • The authors should state which version of the asset is used and, if possible, include a URL.

1004 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.

1005 • For scraped data from a particular source (e.g., website), the copyright and terms of service
1006 of that source should be provided.

1007 • If assets are released, the license, copyright information, and terms of use in the package
1008 should be provided. For popular datasets, `paperswithcode.com/datasets` has curated
1009 licenses for some datasets. Their licensing guide can help determine the license of a dataset.

1010 • For existing datasets that are re-packaged, both the original license and the license of the
1011 derived asset (if it has changed) should be provided.

1012 • If this information is not available online, the authors are encouraged to reach out to the
1013 asset’s creators.

1014 **13. New assets**

1015 Question: Are new assets introduced in the paper well documented and is the documentation
1016 provided alongside the assets?

1017 Answer: **[TODO]**

1018 Justification: **[TODO]**

1019 Guidelines:

1020 • The answer **[N/A]** means that the paper does not release new assets.

1021 • Researchers should communicate the details of the dataset/code/model as part of their sub-
1022 missions via structured templates. This includes details about training, license, limitations,
1023 etc.

1024 • The paper should discuss whether and how consent was obtained from people whose asset is
1025 used.

1026 • At submission time, remember to anonymize your assets (if applicable). You can either create
1027 an anonymized URL or include an anonymized zip file.

1028 **14. Crowdsourcing and research with human subjects**

1029 Question: For crowdsourcing experiments and research with human subjects, does the paper
1030 include the full text of instructions given to participants and screenshots, if applicable, as well as
1031 details about compensation (if any)?

1032 Answer: **[TODO]**

1033 Justification: **[TODO]**

1034 Guidelines:

1035 • The answer [N/A] means that the paper does not involve crowdsourcing nor research with
1036 human subjects.

1037 • Including this information in the supplemental material is fine, but if the main contribution of
1038 the paper involves human subjects, then as much detail as possible should be included in the
1039 main paper.

1040 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or
1041 other labor should be paid at least the minimum wage in the country of the data collector.

1042 **15. Institutional review board (IRB) approvals or equivalent for research with human subjects**

1043 Question: Does the paper describe potential risks incurred by study participants, whether such
1044 risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals
1045 (or an equivalent approval/review based on the requirements of your country or institution) were
1046 obtained?

1047 Answer: **[TODO]**

1048 Justification: **[TODO]**

1049 Guidelines:

1050 • The answer [N/A] means that the paper does not involve crowdsourcing nor research with
1051 human subjects.

1052 • Depending on the country in which research is conducted, IRB approval (or equivalent) may
1053 be required for any human subjects research. If you obtained IRB approval, you should
1054 clearly state this in the paper.

1055 • We recognize that the procedures for this may vary significantly between institutions and
1056 locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines
1057 for their institution.

1058 • For initial submissions, do not include any information that would break anonymity (if
1059 applicable), such as the institution conducting the review.

1060 **16. Declaration of LLM usage**

1061 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-
1062 standard component of the core methods in this research? Note that if the LLM is used only for
1063 writing, editing, or formatting purposes and does *not* impact the core methodology, scientific
1064 rigor, or originality of the research, declaration is not required.

1065 Answer: **[TODO]**

1066 Justification: **[TODO]**

1067 Guidelines:

1068 • The answer [N/A] means that the core method development in this research does not involve
1069 LLMs as any important, original, or non-standard components.

1070 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not be
1071 described.